Future University in Egypt
Faculty of Engineering and Technology

Together

An AI-Based Sign-Language Translator (ASL & ArSL)

A Graduation Project Thesis

By

Abdelfattah	**20225889**
Michael Abdallah	**20213601**
Ahmed Mohamed Nagib	**20170423**
Ahmed Ashraf Shawareb	**20180097**

Submitted to the Department of Computers and Intelligent Systems Engineering
in Partial Fulfilment of the Requirements for the Degree of
Bachelor of Science in Computers and Intelligent Systems Engineering

Supervisor(s):
Prof. Medhat Awadallah

July 2026

Declaration

We confirm that this thesis, presented for the degree of Bachelor of Science in Computers and Intelligent Systems Engineering, has been composed entirely by ourselves, has been solely the result of our own work, and has not been submitted for any other degree or professional qualification. Where the work of others has been used, it has been duly acknowledged and cited in the text.

Author	**Signature**	**Date**
Abdelfattah Moustafa (20225889)		7-7-2026
Michael Abdallah (20213601)		7-7-2026
Ahmed Mohamed Nagib (20170423)		7-7-2026
Ahmed Ashraf Shawareb (20180097)		7-7-2026

Abstract

Everyday communication remains a persistent barrier for deaf and hard-of-hearing individuals. Current assistive tools usually fall short: they are often unidirectional, restricted to a single high-resource sign language, or require specialized hardware.

To solve this, we created **Together**: a real-time, bidirectional sign-language translation system operating entirely in a standard web browser. It supports American Sign Language (ASL) and the comparatively low-resource Arabic/Egyptian Sign Language (ArSL). The system translates signs into text and synthesized speech, and turns text or speech back into sign via a landmark-driven avatar. It also features a live two-person meeting mode, mediating conversations between a signer and speaker over WebRTC.

Under the hood, the pipeline extracts pose, hand, and face landmarks in the browser using MediaPipe Holistic. Server-side, isolated signs are classified by two models: a 250-class TFLite network for ASL and a PyTorch CNN-GRU for the 20-class ArSL set. Recognized signs gather in a voting buffer and are passed as "gloss" to a large language model (Gemini 3.5 Flash with an offline Ollama fallback). This AI reconstructs grammatical sentences, bridging the structural gap between spoken-language syntax and sign-language gloss.

Evaluations confirmed the system's strength. On a within-dataset (70/15/15) split, recognition models achieved **80%** accuracy for ASL and **99.41%** for ArSL. The ASL model also hit **62.4%** accuracy during evaluation on SignASL dataset, confirming generalization to unseen signers. Also, ArSL hit 88% accuracy during signer-independent evaluation. Translation quality against human references reached a BLEU-4 of **0.80** and chrF of **0.91** for ASL, and **0.42** and **0.59** for ArSL, beating raw-gloss chrF baselines of **0.54** and **0.34**. These word- and character-level improvements prove the language model effectively supplies the grammatical structure raw gloss lacks. Ultimately, gloss-mediated, LLM-assisted translation provides a viable, accessible path for bidirectional communication, particularly for under-resourced sign languages.

Contents

List of Figures

List of Tables

List of Acronyms

Acronym	Definition
AI	Artificial Intelligence
API	Application Programming Interface
ArSL	Arabic Sign Language
ASGI	Asynchronous Server Gateway Interface
ASL	American Sign Language
BLEU	Bilingual Evaluation Understudy (translation metric)
BN	Batch Normalization
chrF	Character n-gram F-score (translation metric)
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DFD	Data-Flow Diagram
ECA	Efficient Channel Attention
ECE	Electronics and Communication Engineering
EMA	Exponential Moving Average
ER	Entity-Relationship
FPS	Frames Per Second
GAP	Global Average Pooling
GCN	Graph Convolutional Network
GISLR	Google Isolated Sign Language Recognition
GRU	Gated Recurrent Unit
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World (index)
HoH	Hard-of-Hearing
HTTP	Hypertext Transfer Protocol
I3D	Inflated 3D ConvNet
INT8	8-bit Integer (quantization)
JWT	JSON Web Token
LLM	Large Language Model
LSTM	Long Short-Term Memory
MBConv	Mobile Inverted Bottleneck Convolution

MENA	Middle East and North Africa
MHSA	Multi-Head Self-Attention
ML	Machine Learning
MSA	Modern Standard Arabic
NMM	Non-Manual Marker
ORM	Object-Relational Mapping
PCM	Pulse-Code Modulation
PWA	Progressive Web Application
REST	Representational State Transfer
RTL	Right-to-Left
SBERT	Sentence-BERT (sentence embeddings)
SiLU	Sigmoid Linear Unit (Swish activation)

List of Symbols

Symbol	Meaning
N	Number of frames in a landmark sequence (rolling window, up to 60)
w, h	Captured video frame width and height in pixels
x = (x, y, z)	A single 3-D landmark coordinate (z zeroed for the Arabic model)
c	Shoulder-centre point used to translate-normalize landmarks
s	Shoulder-width scale used to size-normalize landmarks
α	Exponential-moving-average smoothing weight (α = 0.65)
p_i	Soft-max probability of class i
τ	Confidence-acceptance threshold (τ = 0.80 ASL, 0.45 ArSL)
d_cos	Cosine distance between SBERT embeddings (1 − cosine similarity)
σ	Per-clip landmark standard deviation used to normalize the ASL features
$\Delta^{(1)}$, $\Delta^{(2)}$	First- and second-order motion (velocity, acceleration) landmark features
d_k	Self-attention head dimension (d_k = 48)
H	Number of self-attention heads (H = 4)
ε	Label-smoothing coefficient (ε = 0.1)

Chapter 1: Introduction

Communication is a fundamental human right, yet millions of Deaf and hard-of-hearing individuals find it obstructed on a daily basis. The World Health Organization estimates that over 1.5 billion people currently live with some degree of hearing loss—a number expected to grow significantly in the coming decades [1]. For many in this community, their first and most natural language is a signed one, not spoken. Languages like American Sign Language (ASL) and Arabic/Egyptian Sign Language (ArSL) aren't just gestures; they are complete, rule-governed languages with their own unique grammar, structure, and syntax, completely distinct from the spoken languages around them [2]. The divide between the Deaf and hearing worlds isn't a linguistic deficit—it's simply a translation gap. Most hearing people do not know how to sign, and we lack the everyday tools to fluently bridge these two worlds in real time.

While automatic sign-language processing has made impressive strides, today's systems still stumble over three major hurdles [3]. First, they are almost entirely one-way streets: they either convert sign into text, or animate text into sign, but rarely handle both within the same platform. Second, research and development tend to focus almost exclusively on high-resource languages —usually ASL—leaving regional languages like ArSL critically underserved. Finally, many of these systems require bulky, expensive hardware like data gloves or depth cameras, or they only work flawlessly in a controlled lab setting. This makes them highly impractical for everyday, spontaneous use.

To solve this, this thesis introduces Together: a real-time, two-way, and bilingual sign-language translation platform that runs entirely in a standard web browser. It requires zero installation and uses nothing more than an everyday webcam. Together supports both ASL and ArSL, seamlessly translating in both directions—turning signs into text and spoken audio, while animating text or speech back into sign language via a responsive, landmark-driven avatar. Beyond standalone translation, it features a live meeting mode where a signer and a speaker can hold a natural, face-to-face conversation mediated by a peer-to-peer video connection. These capabilities are delivered through eight modules — HandScript, VoiceBridge, SignType, TalkSide and SignLine

for translation, plus Dictionary, Practice and Analytics for reference and learning — within one bilingual web application.

1.1 Motivation

Socially, accessible communication has a direct effect on the education, employment, and independence of Deaf individuals. A tool that lowers the barrier between signers and non-signers therefore carries tangible human value. This need is especially pronounced in the Arabic-speaking world: Arabic Sign Language is used by a large population, yet it remains a low-resource language in computational terms, with few public datasets and almost no fluent, bidirectional translation systems. Treating ArSL as a first-class language alongside ASL — rather than as an afterthought — addresses a genuine and under-explored need.

Technically, recent advances make a practical solution feasible for the first time. Browser-based landmark extraction removes the dependence on specialised cameras and allows sign articulation to be captured on ordinary consumer devices [4]. Lightweight neural inference makes real-time recognition possible on commodity hardware. Most importantly, the emergence of capable large language models offers, for the first time, a robust way to transform the terse, grammatically distinct gloss produced by sign recognition into fluent, natural-language sentences without relying on brittle hand-crafted rules [5]. The convergence of these technologies enables a system that is simultaneously real-time, bidirectional, bilingual, and universally deployable through a web browser — a combination not previously demonstrated.

1.2 Aims and Objectives

The core aim of *Together* is to design, build, and evaluate a real-time, two-way sign-language translation system that operates entirely within a standard web browser. Crucially, it bridges the current technological divide by supporting both a widely researched language (ASL) and a historically under-resourced one (ArSL).

To bring this vision to life, we set out to achieve the following key objectives:

- **Build an accessible front-end:** Develop a browser-based tool that uses landmark extraction to capture live sign movements instantly, requiring no specialized cameras or hardware.
- **Train robust recognition models:** Train and rigorously test sign-recognition models for both ASL and ArSL, ensuring they genuinely generalize to new, unseen signers rather than just memorizing training data.
- **Bridge the grammar gap:** Design a translation stage where a large language model transforms raw, recognized signs (gloss) into fluent, natural spoken-language sentences, complete with a reliable offline fallback.
- **Enable reverse translation:** Bring the system full circle by turning text and spoken words back into sign language using the semantic retrieval of sign animations.
- **Unify the experience:** Seamlessly integrate all these moving parts into a single, bilingual web application, featuring a live, two-person meeting mode for natural conversations.
- **Measure the impact:** Quantitatively evaluate the entire platform—focusing on recognition accuracy and translation quality—to clearly demonstrate exactly how much the AI language model improves the final translation.

1.3 Proposed Approach

Together seamlessly weaves computer vision, machine learning, and natural language processing into one unified pipeline. This high-level architecture is outlined in Fig. 1.1. The client operates entirely within the web browser, where MediaPipe Holistic instantly captures pose, hand, and face landmarks from a standard webcam feed [4] and streams them server-side. Behind the scenes, a FastAPI back-end smoothly routes requests, runs model inference, and manages real-time communication. Meanwhile, a PostgreSQL database equipped with a vector extension stores user data alongside the sign animations needed for the avatar.

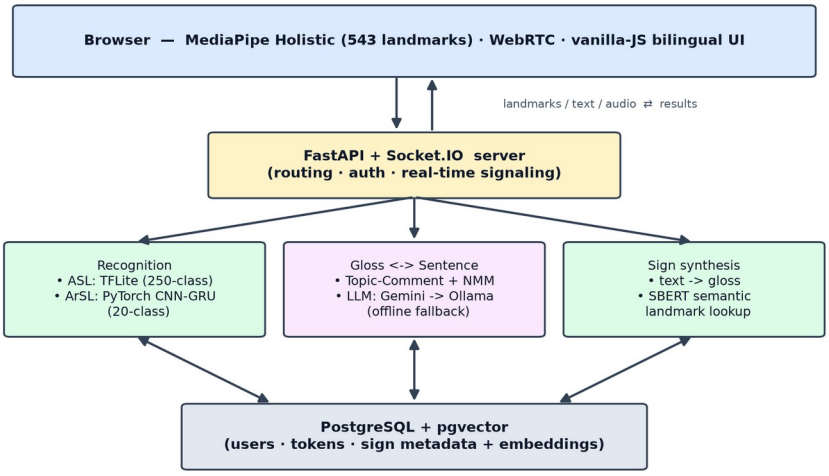

Figure 1.1: High-Level Architecture of the Together system

The two distinct translation pathways are illustrated in Fig. 1.2. When translating from sign to text or speech, the incoming landmark stream is classified into specific signs by one of two AI models: a network for the ASL vocabulary and a recurrent network for ArSL. Because these models recognize isolated signs, their outputs gather in a voting buffer to form a raw sequence of words (gloss). This sequence is then handed to a large language model, which reconstructs it into a natural, grammatically correct sentence and optionally reads it aloud via text-to-speech.

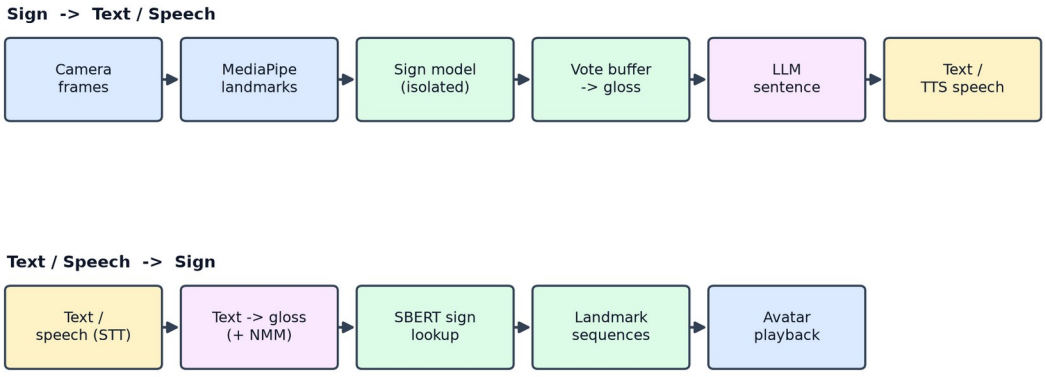

Figure 1.2: The bidirectional translation pipeline

For the reverse journey—from speech or text back to sign—spoken input is transcribed, mapped to gloss, and paired with stored sign animations via semantic embedding search, which the digital avatar then plays back. Crucially, the entire system is designed to degrade gracefully to offline components, ensuring *Together* remains fully functional even if cloud connectivity drops.

These two directions are combined in the live meeting mode, shown in Fig. 1.3, which lets a signer and a speaker hold a real-time conversation. The two participants join a shared room and exchange audio and video over a peer-to-peer connection, with the server coordinating the session and performing translation. The signer's articulation is processed through the sign-to-text/speech pipeline and delivered to the speaker as live captions and synthesised speech, while the speaker's voice is processed through the speech-to-sign pipeline and presented to the signer as a sign avatar. The two pipelines therefore run concurrently, one per participant, so that each person receives the conversation in their own modality.

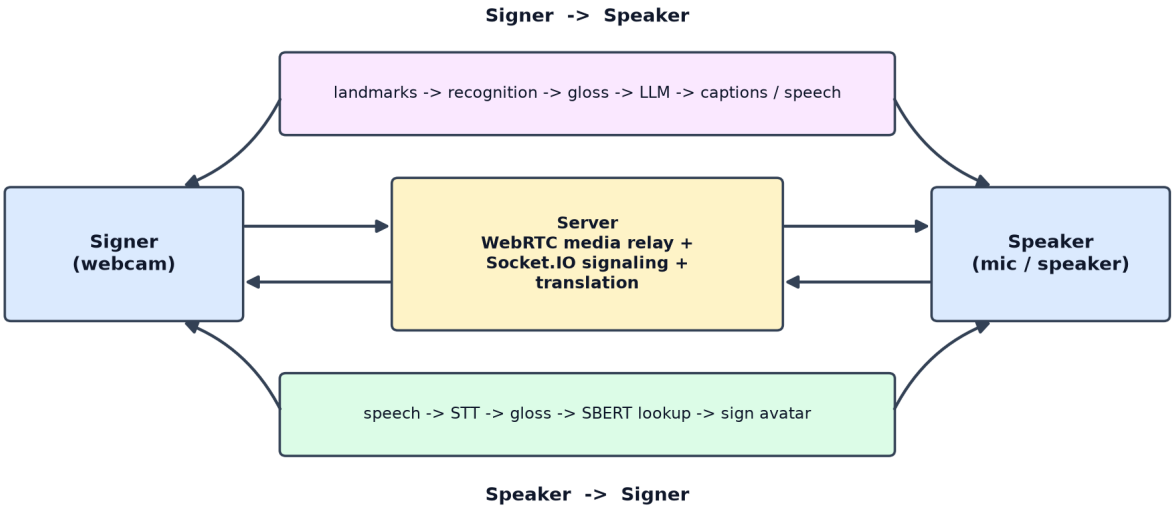

Figure 1.3: The live two-person meeting pipeline

1.4 Thesis Organization

The remainder of this thesis explores the system's complete development:

- **Chapter 2:** Reviews sign-language linguistics, previous translation research, and existing ArSL resources.
- **Chapter 3:** Outlines our system analysis, core requirements, datasets, and architectural blueprint.
- **Chapter 4:** Details the technical implementation, including the recognition models, AI translation, sign synthesis, and real-time infrastructure.
- **Chapter 5:** Evaluates the platform's accuracy, translation quality, and real-world behaviour.
- **Chapter 6:** Summarizes our key achievements and highlights directions for future work.

Chapter 2: Review of Literature

2.1 Technical Background

Sign-language translation decomposes into two directions, and the systems reviewed later differ chiefly in which they implement and how. The sign-to-spoken direction begins with recognition. Research benchmarks such as the Word-Level ASL dataset have shown that recognition models split into appearance-based networks — two-dimensional CNNs with recurrent aggregation, or three-dimensional CNNs such as I3D — and pose-based networks that operate on extracted skeleton keypoints, of which graph convolutional networks are the strongest; on a 2,000-sign vocabulary both families remain below ~35% top-1 accuracy, but pose-based models match appearance models at far lower cost [6]. Microsoft's MS-ASL provides a comparably large American benchmark of 1,000 signs recorded from 222 signers in unconstrained conditions [7], while the Turkish AUTSL corpus added depth and skeleton modalities for 226 signs [8]. The skeleton-aware graph approach later won the signer-independent recognition challenge on AUTSL with around 98% accuracy, confirming that keypoint models generalise to unseen signers when training data is diverse [9].

The **spoken-to-sign** direction, and full translation, was formalised by Camgöz et al., who paired a CNN visual encoder with an attention-based recurrent encoder–decoder and used gloss as an intermediate representation [10], later replacing it with a transformer trained jointly for recognition and translation via a CTC loss, which roughly doubled quality on the German PHOENIX14T corpus [11]. Comparable continuous corpora such as How2Sign offer many hours of aligned ASL video and English text [12], but these resources exist only for high-resource languages. They are powerful yet depend on large parallel corpora of continuous signing — a resource that does not exist for Arabic Sign Language, whose datasets are small, recorded from few signers, and often limited to the alphabet (Table 2.1) [13], [14], [15]. Finally, all of these pipelines terminate in gloss, which is not a grammatical sentence: sign languages use space, non-manual markers, and a Topic–Comment order that differ from spoken syntax [16], [17], so fluent

output requires a dedicated language-generation stage. These three facts — the cost of recognition, the corpus dependence of translation, and the grammar gap — are the lenses through which the following systems are assessed.

Table 2.1: Comparison of different Datasets

Dataset	Language	Level	Size	Signers	Modality
WLASL [6]	ASL	Isolated (word)	~21,000 videos, 2,000 glosses	100+	RGB video
MS-ASL [7]	ASL	Isolated (word)	~25,000 videos, 1,000 signs	222	RGB video
AUTSL [8]	Turkish SL	Isolated (word)	38,336 samples, 226 signs	43	RGB, depth, skeleton
How2Sign [12]	ASL	Continuous (sentence)	80+ hours	11	RGB, depth, pose, speech
PHOENIX14T [10]	German SL	Continuous (sentence)	8,257 sentences, 1,066 glosses	9	RGB video
KArSL [13]	ArSL	Isolated (word)	502 signs, 75,300 samples	3	RGB, depth, skeleton
ArASL/ ArSL2018 [14]	ArSL	Alphabet (static)	54,049 images, 32 classes	40	Grayscale image

2.2. Existing Applications and Systems

2.2.1 SignAll

SignAll [18] translates American Sign Language into English text at kiosks and in educational tools, and is one of the few systems to attempt sign-to-text rather than the easier reverse direction.

- **Dataset.** A large proprietary corpus of ASL recorded from native Deaf signers, built in-house to train the recogniser; it is not publicly available.
- **Architecture.** A multi-camera rig (typically three or more cameras) combined with colour-marked gloves for precise finger tracking; computer-vision modules perform hand-shape and motion recognition, and a rule-based linguistic component maps the recognised manual and non-manual features to English grammar.
- **Limitation.** The dependence on multiple calibrated cameras and gloves makes it non-portable and impossible to run in a browser; it is unidirectional (sign → text) and ASL-only, and its proprietary dataset prevents reproduction.

2.2.2 Hand Talk

Hand Talk [19] is a widely adopted mobile application and website plug-in that converts written and spoken language into sign through a three-dimensional avatar (Hugo), for ASL and Brazilian Sign Language (Libras).

- **Dataset.** Rather than a recognition dataset, it relies on a curated dictionary of signs animated by human translators, paired with a text-processing engine; the translation component is reported to combine linguistic rules with machine learning.
- **Architecture.** A natural-language layer maps input text to a gloss sequence, which a 3D character-animation engine renders by playing and blending pre-built sign animations.
- **Limitation.** It is strictly one-directional (spoken → sign) and cannot interpret a user's own signing; output is bounded by the curated animation vocabulary, and avatar comprehensibility is a known constraint for fluent signers.

2.2.3 SLAIT

SLAIT [20] demonstrated real-time American Sign Language recognition running directly in a web browser from an ordinary webcam, and is the closest competitor in deployment model to *Together*.

- **Dataset.** A self-collected set of recorded signs; public demonstrations covered only a limited vocabulary of words and phrases.
- **Architecture.** In-browser landmark/keypoint extraction (MediaPipe-style) followed by a deep sequence model — a recurrent or transformer network over the keypoint stream — producing text in real time.
- **Limitation.** Restricted vocabulary, ASL-only, and unidirectional (sign → text); as an early-stage start-up product its availability and robustness in the wild are uncertain.

2.2.4 KinTrans

KinTrans [21] converts signing into text and voice for enterprise settings such as service counters, and is notable for supporting Arabic Sign Language alongside ASL.

- **Dataset.** A large proprietary multi-signer corpus recorded by the company, reportedly spanning thousands of signs and including Arabic content; not public.
- **Architecture.** A three-dimensional depth camera captures skeletal joint positions, and a machine-learning classifier recognises signs from the resulting body- and hand-joint motion sequences.
- **Limitation.** It requires dedicated 3D-camera hardware and a fixed installation, so it is neither mobile nor browser-based; it is unidirectional (sign → spoken) and its data is closed.

2.2.5 Signapse

Signapse [22] generates photorealistic sign-language video from text for fixed contexts such as transport announcements and website content, in British and American Sign Language.

- **Dataset.** A proprietary corpus of professionally recorded signer footage used to train its generative models.

- **Architecture.** Deep generative models (GAN-based video synthesis) produce continuous, photorealistic signing; a text-to-gloss stage drives which signs are generated and stitched.
- **Limitation.** It is one-directional (spoken → sign), oriented to pre-rendered, domain-specific announcements rather than open conversation, and does not interpret a user's signing

2.3 Comparison with Together

Three core differences set *Together* apart, directly addressing the critical gaps left by existing systems:

- **It is fully bidirectional.** Every other product we reviewed commits to just one direction. Tools like SignAll, SLAIT, and KinTrans can interpret sign language but cannot produce it, while Hand Talk and Signapse generate signs but cannot interpret them. *Together* handles both. Furthermore, its live meeting mode runs these processes concurrently, empowering a signer and a speaker to have a genuine, back-and-forth conversation—a feature none of the other five systems offer.
- **It champions both high- and low-resource languages.** While KinTrans does include Arabic, it only translates from sign to spoken language and requires closed hardware. Together elevates Arabic Sign Language (ArSL) to a first-class language alongside ASL, seamlessly translating it in both directions. Rather than sidestepping the data-scarcity problem documented in Table 2.1, Together tackles it head-on.
- **It is hardware-free and reproducible.** SignAll demands a calibrated multi-camera rig and data gloves, and KinTrans requires a depth camera. While SLAIT shares our browser-and-webcam model, it only works in one direction. *Together* runs effortlessly in an ordinary web browser. Because it relies on public datasets and an LLM-driven gloss-to-text pipeline, it is readily deployable without specialized equipment and—unlike proprietary products—fully reproducible.

2.4 Comparative Analysis

Table 2.2: Comparison of existing sign-language systems with Together.

System	Direction	Languages	Dataset	Architecture	Key limitation
SignAll [18]	Sign → Spoken	ASL	Proprietary ASL corpus	Multi-camera CV + marker gloves + rule-based NLP	Needs camera rig + gloves; not portable
Hand Talk [19]	Spoken → Sign	ASL, Libras	Curated animation dictionary	Text→gloss + 3D avatar	One-directional; fixed vocabulary
SLAIT [20]	Sign → Spoken	ASL	Self-collected (small)	Webcam landmarks + deep sequence model	Limited vocabulary; one-directional
KinTrans [21]	Sign → Spoken	ASL, ArSL	Proprietary multi-signer corpus	3D-camera skeleton + ML classifier	Needs 3D-camera hardware; one-directional
Signapse [22]	Spoken → Sign	BSL, ASL	Proprietary signer video	Deep generative (GAN) video	One-directional; domain-limited
Together (this work)	*Bidirectional*	*ASL, ArSL*	*Public datasets*	*Landmarks + isolated models + LLM gloss→sentence*	*Isolated (not continuous) signing*

Chapter 3: System Analysis and Design

This chapter presents the analysis and design of *Together* through a complete set of models: the requirements and system context, the architecture, the static structure (class and data models), the dynamic behaviour (data-flow, activity, and state models), the interaction (sequence) diagrams for each major scenario, and the datasets used. The detailed design of the two recognition models is deferred to Chapter 4.

3.1 Development Process and Methodology

The project followed an **iterative, incremental** process. Rather than specifying the whole system up front, the team built a thin end-to-end slice first — webcam capture, a single recognition path, and a minimal UI — and then grew it feature by feature, hardening each layer once it worked. The work proceeded in clear phases: a research and data phase; a recognition-model phase for ASL and then ArSL; a backend-services phase (gloss, providers, sign lookup); a frontend and bilingual-UI phase; a real-time meeting phase; and finally a hardening phase covering authentication, latency profiling and deployment.

Table 3.1: Project development phases and milestones.

Phase	Period	Milestone / deliverable
1. Research & datasets	Oct 2025–Jan 2026	Literature review; landmark representation; sign-video corpus
2. ASL recognition	Jan 2026–Feb 2026	250-class TFLite pipeline; live capture & voting
3. ArSL recognition	Feb 2026	20-class CNN-GRU model; bilingual switch
4. Language & lookup	March–Apr 2026	Gloss engine + NMM; SBERT/pgvector sign lookup; providers
5. Frontend & RTL UI	Apr 2026	Dashboards (EN/AR), avatar player, design system
6. Meetings & realtime	Apr–May 2026	WebRTC mesh; Socket.IO streaming recognition
7. Evaluation & hardening	May 2026	Baseline eval (Top-1 0.80); auth; latency profiling; deploy
8. Documentation	Jun–Jul 2026	Thesis, diagrams, final submission

3.2 Requirements and Analysis

The requirements were derived from the project aim (Section 1.2) and refined during the iterative process. They are grouped into functional and non-functional requirements, followed by the hardware and software needed to run the system, and the principal use cases.

3.2.1 Functional Requirements

Table 3.2: Functional requirements.

	Requirement	**Realised by**
FR1	Capture webcam video and extract body/hand/face landmarks in the browser	MediaPipe Holistic
FR2	Recognise isolated ASL signs from a landmark sequence	ASLService.predict_sign
FR3	Recognise isolated ArSL signs from a landmark sequence	SignLanguagePredictor
FR4	Debounce per-frame predictions into stable sign tokens	Vote/cooldown buffer
FR5	Convert recognized gloss into a natural sentence	gloss_to_sentence
FR6	Convert a typed/spoken sentence into sign gloss + NMM	english_to_gloss, annotate_nmm
FR7	Host a live two-party signer–speaker meeting	WebRTC + Socket.IO
FR8	Support English (LTR) and Arabic (RTL) interfaces	Jinja2 templates
FR9	Authenticate users and protect the API	JWT, rate limits

3.2.2 Non-Functional Requirements

Table 3.3: Non-functional requirements.

ID	**Quality**	**Target / mechanism**
NFR1	Real-time responsiveness	Per-frame streaming at ~20 fps; sign accepted in a few hundred ms; blocking work off the event loop
NFR2	Concurrency	No endpoint blocks the event loop; thread-locked single-instance models; DB pool (size 5, overflow 10)
NFR3	Offline resilience	Provider fallback chains (Gemini → Ollama / pyttsx3 / Whisper); rule-based gloss fallback
NFR4	Security	Argon2id hashing; JWT with rotating refresh tokens; sliding-window rate limiting; security headers
NFR5	Portability	Browser client (no install); Dockerised server; runs on CPU only
NFR6	Accessibility & i18n	Bilingual EN/AR, RTL mirroring, reduced-motion & focus-visible support
NFR7	Observability	Per-stage latency metrics
NFR8	Privacy	Only landmarks (not raw video) are sent to the server for recognition

3.2.3 Hardware Requirements

Table 3.4: Hardware requirements.

Component	Minimum	Notes
Client device	Any laptop/desktop/phone with a webcam and a modern browser	MediaPipe runs via WebAssembly; GPU optional
Camera	Standard 480p+ webcam	Capture configured at ~480×360, 15–20 fps
Server CPU	2+ cores	Inference is CPU-only (TFLite XNNPACK; PyTorch INT8)
Server RAM	~2–4 GB	SBERT + two models + Postgres client
GPU	Not required	All inference runs on CPU
Network	Broadband / Wi-Fi	Socket.IO streaming + optional cloud LLM/TTS/STT

3.2.4 Use Cases

Figure 3.1 shows the use-case diagram with the two human actors — a Deaf/HoH user and a hearing user — and the system/provider actor that fulfils language tasks. Table 3.5 describes the principal use cases.

Figure 3.1: Use-case diagram of the Together platform.

Table 3.5: Principal use cases of the Together platform.

Use case	Actor	Description
Sign → Text (HandScript)	Deaf/HoH user	Sign to the webcam; the system recognises signs and shows a natural sentence
Sign → Speech (VoiceBridge)	Deaf/HoH user	As above, then the sentence is spoken aloud via TTS
Text → Sign (SignType)	Hearing user	Type a sentence; the avatar signs it (gloss → landmark lookup)
Speech → Sign (TalkSide)	Hearing user	Speak; STT transcribes, then the avatar signs the text
Live meeting (SignLine)	Both	Two-party WebRTC call with live captions / sign avatar per role
Practice / Dictionary	Deaf/HoH user	Browse the recognised vocabulary and practise individual signs
Register / Login	Both	Create an account and authenticate to use the API
Switch language	Both	Toggle the EN (LTR) and AR (RTL) interface and model

3.3 System Design and Architecture

Together is a client–server web application. The browser is responsible for capture, landmark extraction, the avatar and the WebRTC media path; the server hosts the recognition models, the language layer, the sign database and the provider chains. Figure 3.2 gives the high-level architecture. Four design decisions anchor the system: the frontend is server-rendered HTML/CSS/JS with no SPA framework or build step; the machine-learning stack uses TFLite for the English model and PyTorch for the Arabic model, with MediaPipe Holistic extracting landmarks in the browser; real-time communication uses Socket.IO for both streaming recognition and WebRTC signalling; and persistence uses PostgreSQL with the pgvector extension for semantic search.

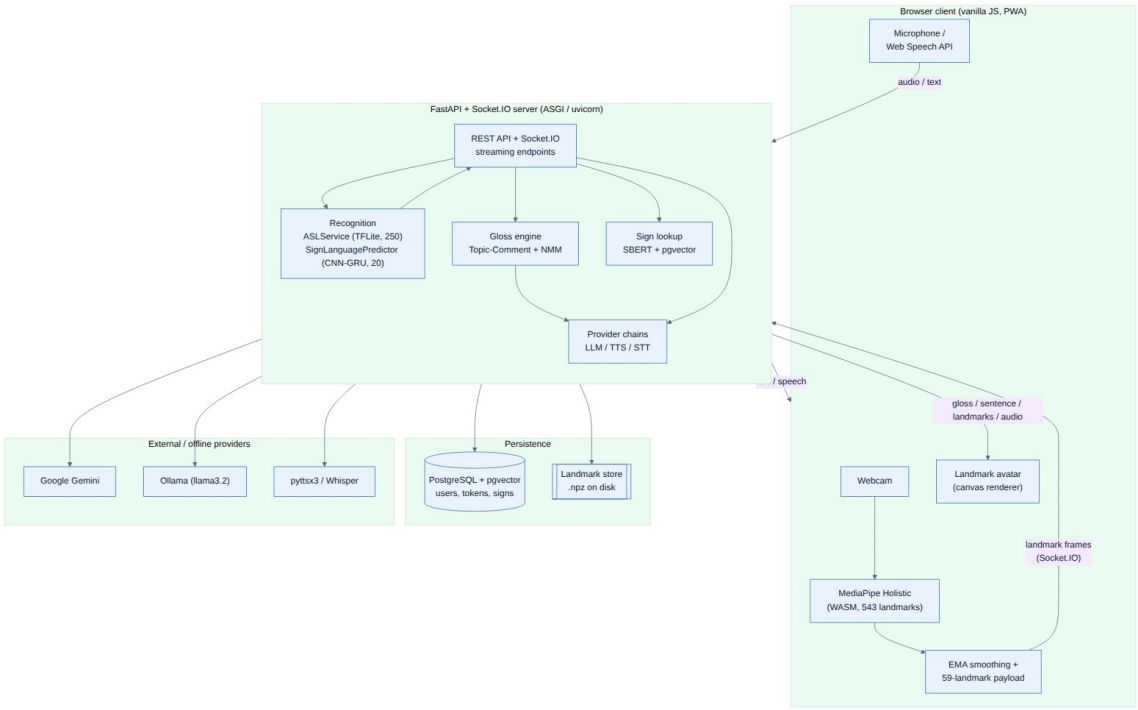

Figure 3.2: High-level system architecture of the Together platform.

The context diagram frames the system as a single process exchanging data with two human actors and the external Gemini API, and the Level-1 data-flow diagram (Figure 3.4) decomposes it into the six functional processes that mirror the use cases.

Figure 3.3: Context diagram (Data-Flow Diagram, Level 0).

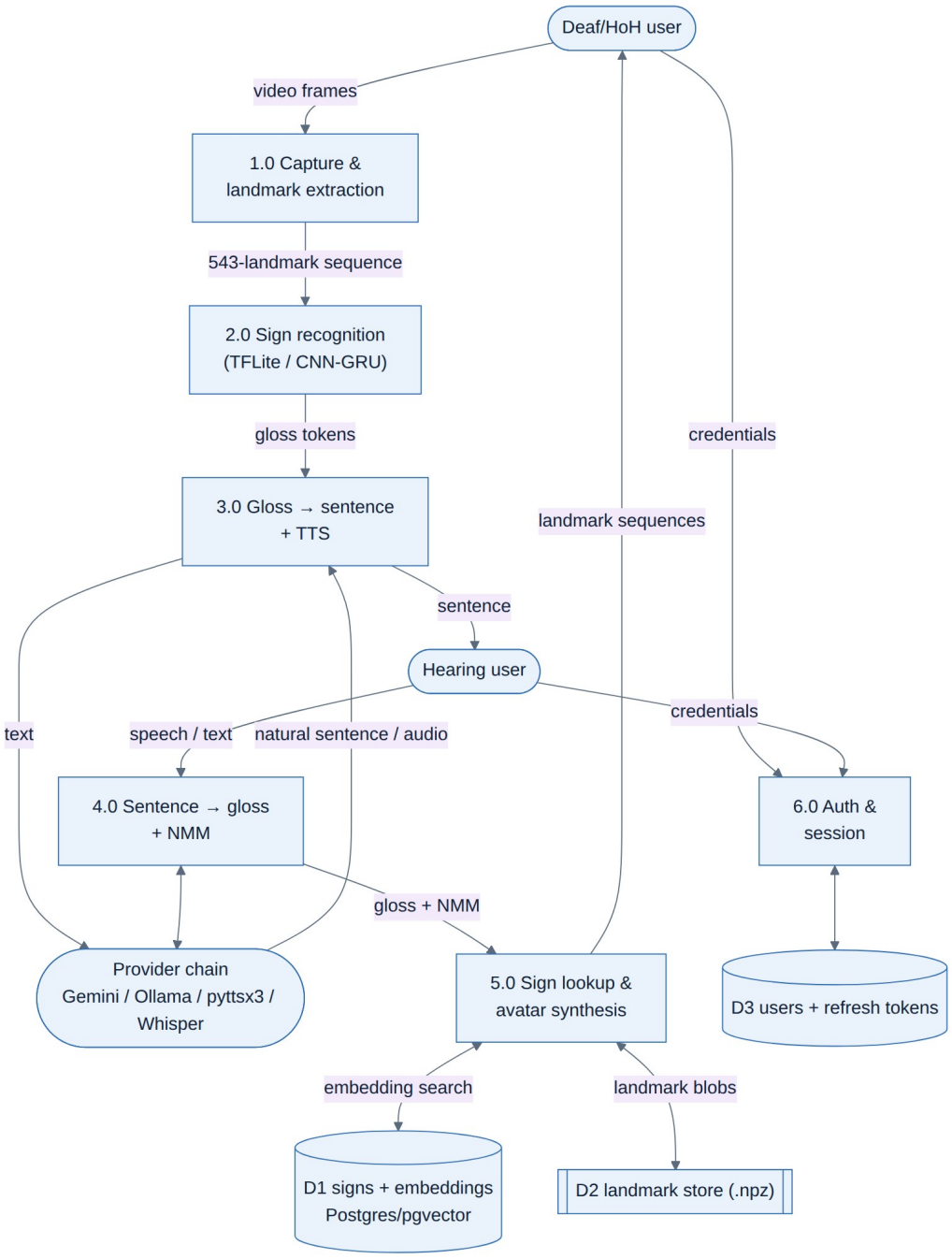

Figure 3.4: Data-Flow Diagram (Level 1).

Internally the server is organised into cooperating components: the FastAPI/Socket.IO entry point routes requests to the recognition service, the gloss engine, the sign-lookup layer and the

provider chains, while the database package and the on-disk landmark store provide persistence. The deployment topology shows the browser talking to a containerised web service that is backed by PostgreSQL and an optional local Ollama service, with Google Gemini reached over HTTPS as the default cloud provider; in a meeting, media flows peer-to-peer over a WebRTC mesh while only signalling passes through the server.

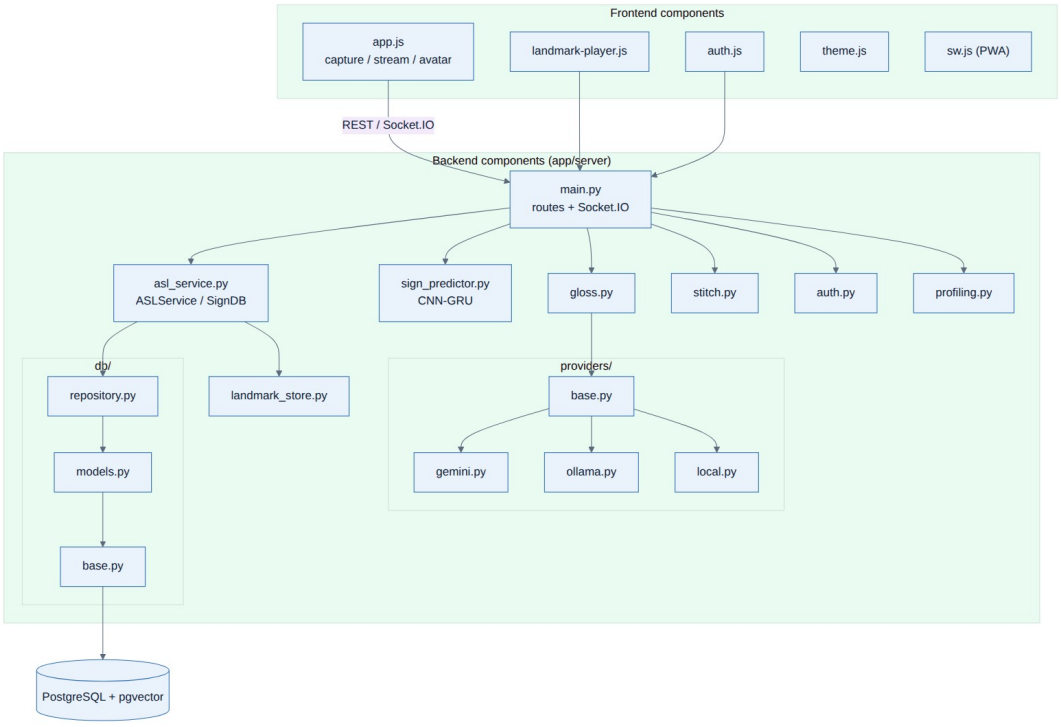

Figure 3.5: Component diagram of the backend and frontend.

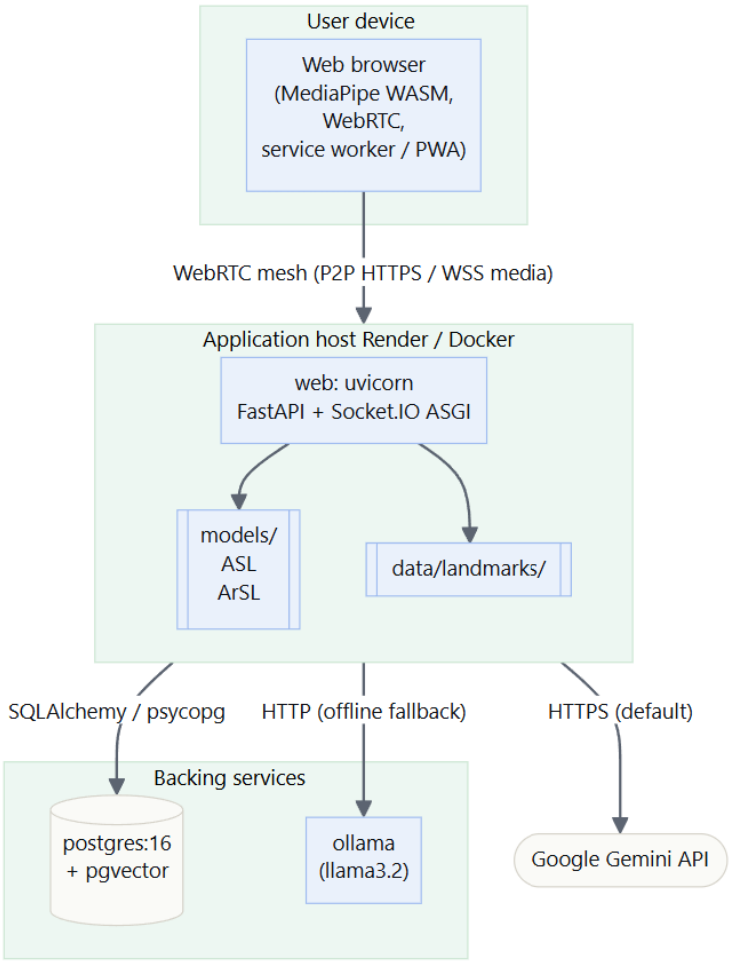

Figure 3.6: Deployment diagram.

3.3.1 UML Models

The static structure of the recognition and lookup subsystem is shown in the class diagram (Figure 3.7). SignDB encapsulates SBERT-backed semantic lookup over PostgreSQL; ArabicSignDB and ASLService specialise it — the former for the 20-word Arabic vocab, the latter by adding the TFLite gesture model — and SignLanguagePredictor wraps the PyTorch CNN-GRU used for Arabic recognition.

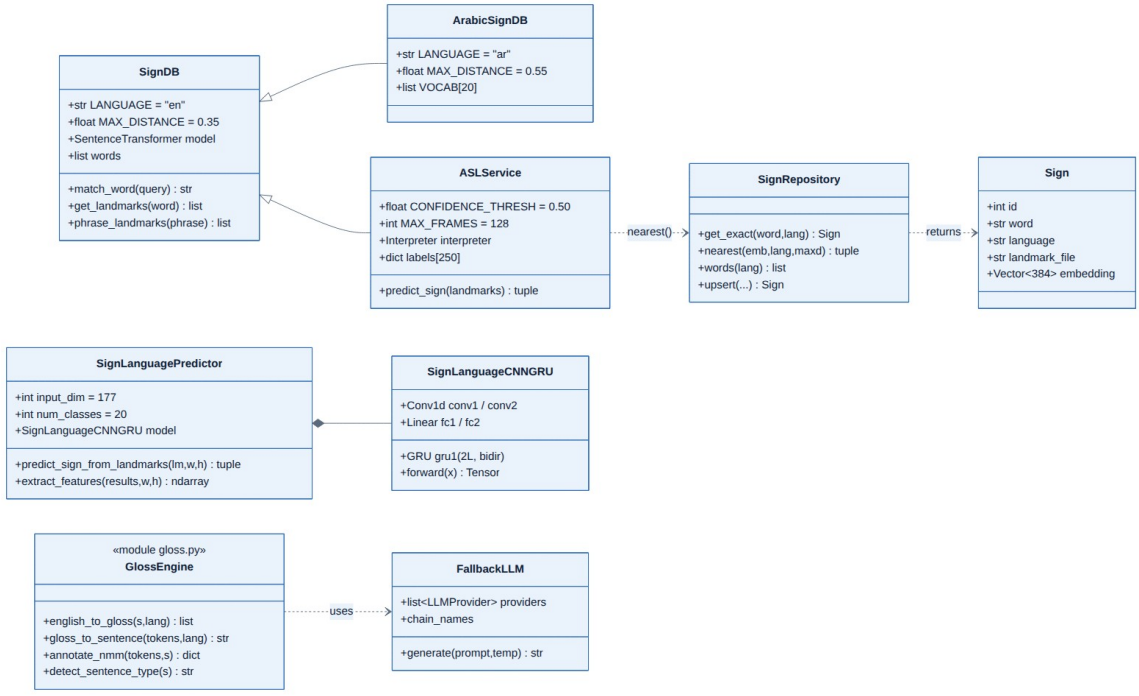

Figure 3.7: Class diagram of the recognition and sign-lookup subsystem

The dynamic behaviour of the principal real-time path — a signer producing text — is shown in the sequence diagram (Figure 3.8) and the activity diagram (Figure 3.9). The browser opens a streaming session, sends one landmark frame per tick, and the server maintains a rolling buffer, runs inference in a worker thread, votes over recent predictions and emits an accepted sign; a separate request then turns the accumulated gloss into a sentence. The capture/recognition logic itself is a small state machine (Figure 3.10) cycling through idle, buffering, inferring, voting and cooldown states.

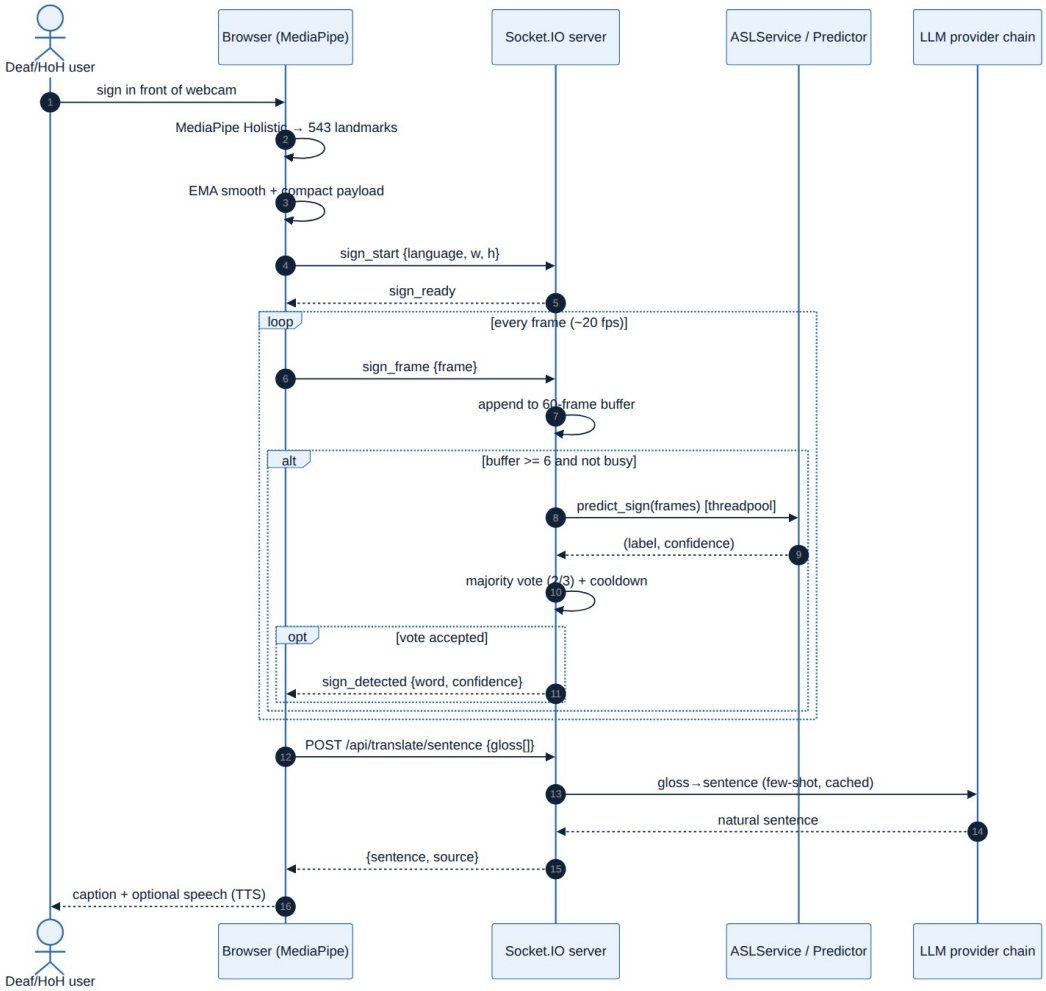

Figure 3.8: Sequence diagram: real-time sign-to-text request flow

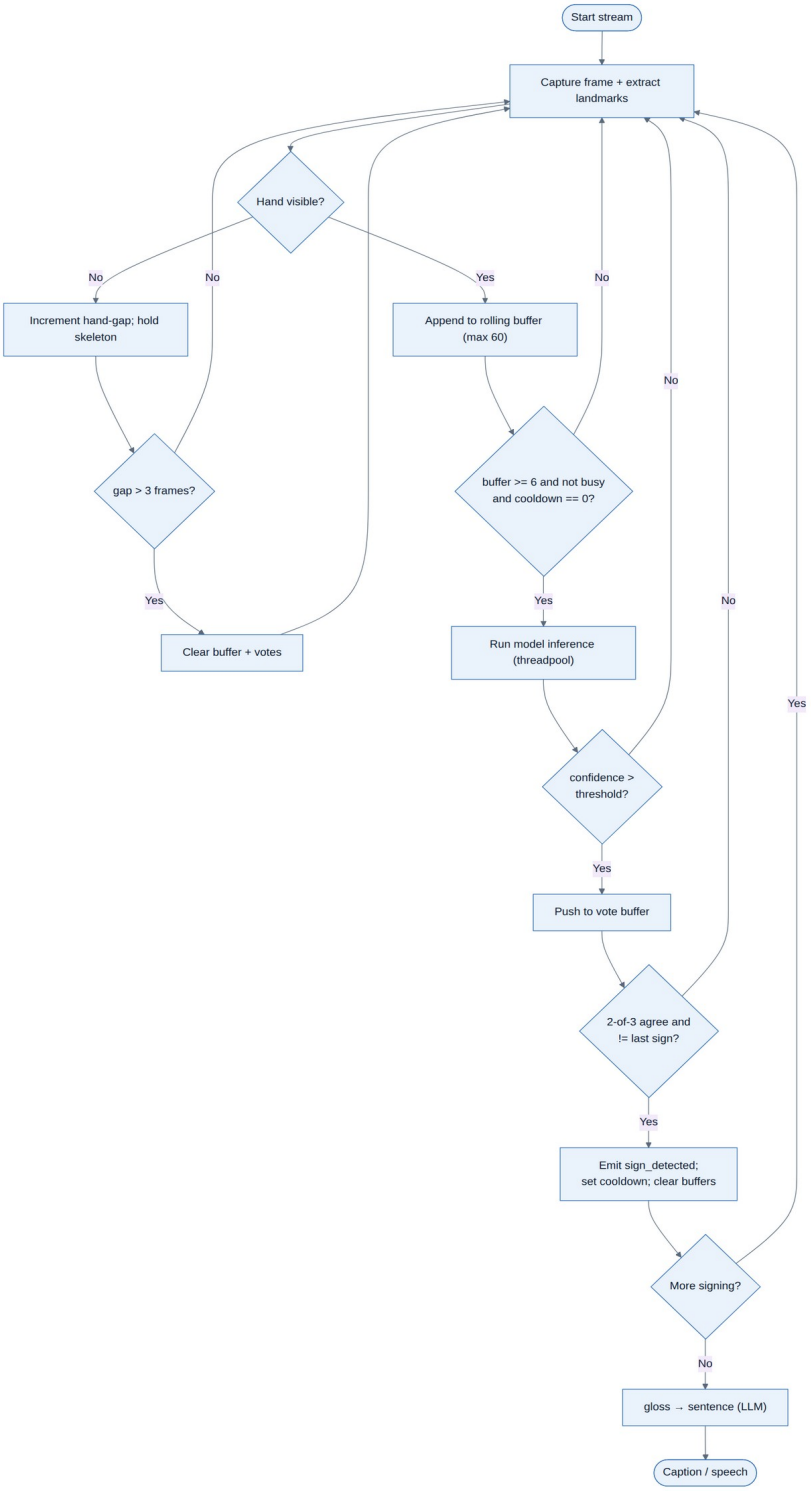

Figure 3.9: Activity diagram of the streaming recognition workflow

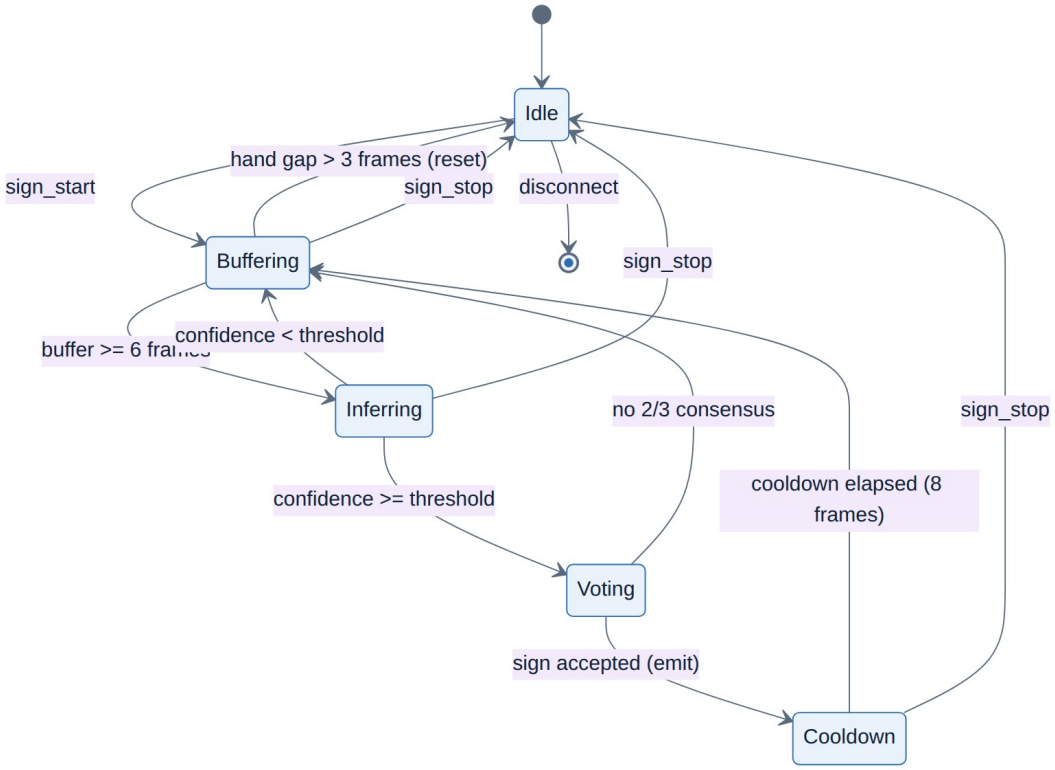

Figure 3.10: State-machine diagram of the capture/recognition states

3.3.2 Data Model

The persistent state is modelled by three entities, shown in Fig. 3.11: users (credentials stored as a hash), refresh_tokens (hashed, expiring, one-to-many with users), and signs (per-language sign metadata with a vector embedding queried by semantic similarity through pgvector).

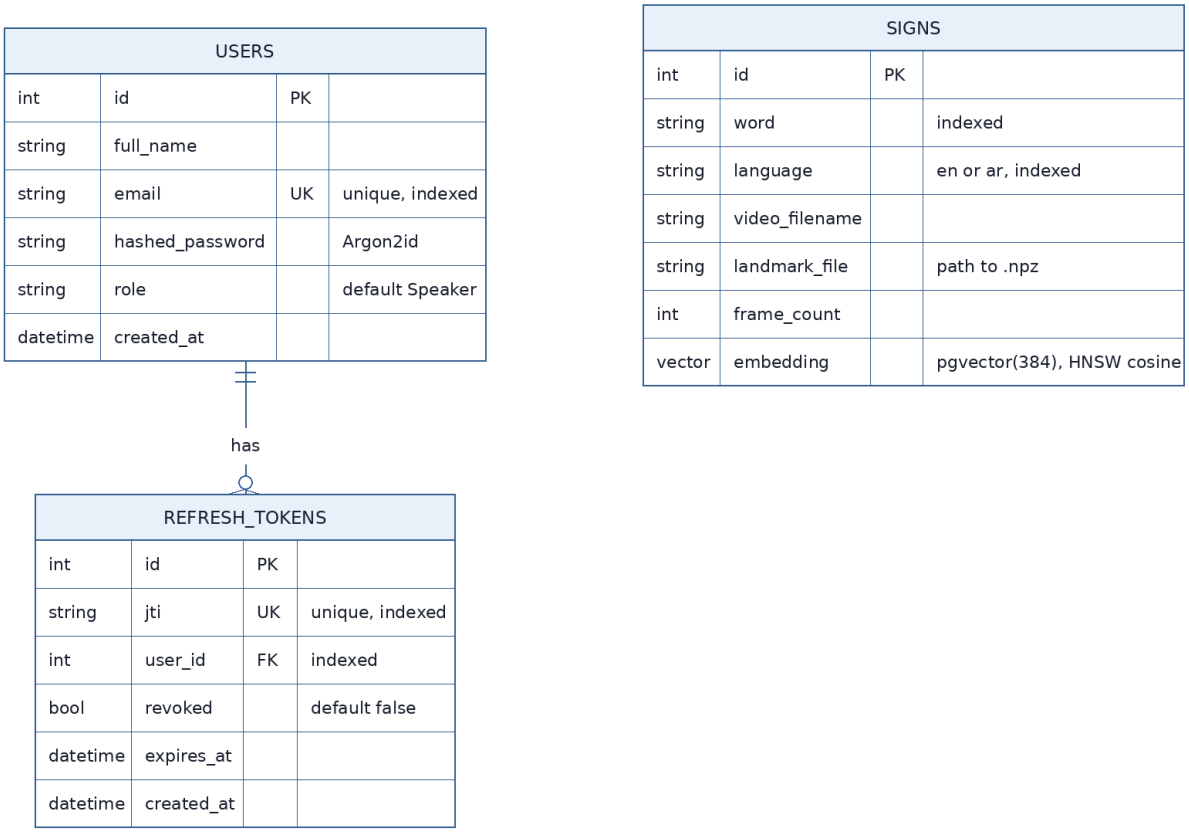

Figure 3.11: Entity-Relationship diagram of the PostgreSQL schema

3.4 Datasets

The two recognition models are trained on **public** datasets. The ASL model uses the **Google Isolated Sign Language Recognition** dataset [23] (~100,000 landmark sequences, 250 signs, 21 Deaf signers); a working subset of 4,078 sequences is split 70/15/15 for training, validation, and test. The ArSL model uses the **Arabic Sign Language 20-Words dataset** of Balaha [24] (8,437 samples, 20 signs, 72 signers), split 70/15/15.

Table 3.6: Datasets used to train the two recognition models.

Dataset	Language	Classes	Samples	Signers	Representation
Google ISLR	ASL	250	~100,000	21	MediaPipe landmarks (543 pts)
Balaha ArSL-20	ArSL	20	8,437	72	MediaPipe landmarks (59 pts)

Chapter 4: Implementation

This chapter describes the implementation of *Together*, concentrating on the two recognition models — the heart of the system — and then the translation, synthesis, and real-time integration that surround them. Every model is documented from input representation through architecture, augmentation, and training. It closes by presenting the eight user-facing modules and the bilingual interface through which all of this reaches the user (Section 4.9).

4.1 Technology Stack

The system is implemented in Python with a FastAPI + Socket.IO back-end and a server-rendered, vanilla-JavaScript front-end. Landmarks are extracted in the browser with MediaPipe Holistic, which builds on the on-device hand-tracking models of MediaPipe [25]. The ASL model runs through the TensorFlow Lite (LiteRT) interpreter; the ArSL model runs in PyTorch. Semantic sign retrieval uses Sentence-BERT [26] over a PostgreSQL + pgvector store, and language generation uses Gemini [27] with a local LLaMA-based Ollama fallback [28].

4.2 The ASL Recognition Model

4.2.1 Input and preprocessing

The end-to-end shape flow of the ASL model is shown in Fig. 4.1. The network core operates on a feature tensor of shape (L, C); at inference the deployed TFLite model instead accepts the **raw** MediaPipe landmarks of shape (60, 543, 3) and performs all preprocessing inside the graph, so the browser only sends raw coordinates.

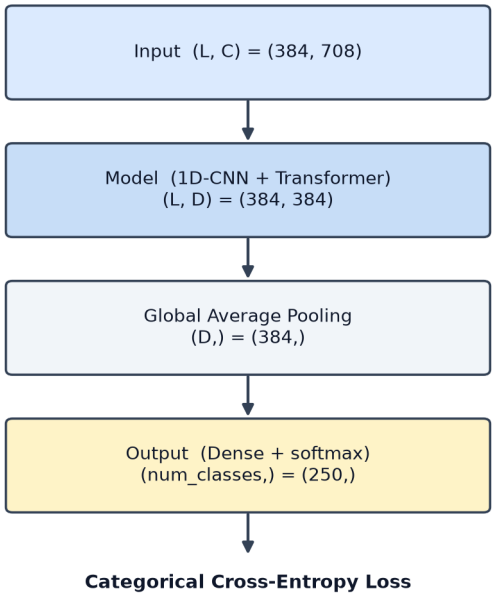

Figure 4.1: ASL model

The preprocessing stage (Fig. 4.2) selects **118** of the 543 landmarks (lips, both hands, and upper body), **drops the z axis** to keep only (x, y), centres the coordinates on the nose landmark, and normalises by the sequence standard deviation. First- and second-order motion features (dx, dy and dx^2, dy^2) are concatenated with the coordinates, yielding **708 features per frame** (118 × 6). During training, sequences are padded to a maximum length of 384, giving a feature tensor of shape (384, 708).

Formally, with p_t the (x, y) landmark vector at frame t and p_nose the nose landmark, the per-clip standardization and motion features are:

$$\tilde{p}_{t,i} = p_{t,i} - p_{t,nose}$$

$$\hat{p}_{t,i} = \frac{\tilde{p}_{t,i}}{\sigma}, \sigma = \sqrt{1/|V| \sum_{(t,i) \in V} \| \tilde{p}_{t,i} \|^2}$$

$$\Delta_t^{(1)} = \hat{p}_t - \hat{p}_{t-1}, \Delta_t^{(2)} = \hat{p}_{t+1} - 2\hat{p}_t + \hat{p}_{t-1}$$

$$f_t = [\hat{p}_t \| \Delta_t^{(1)} \| \Delta_t^{(2)}] \in \mathbb{R}^{708}$$

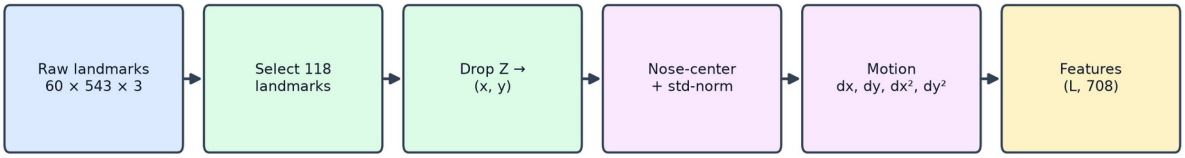

Figure 4.2: ASL preprocessing

4.2.2 Architecture

The model adapts the Squeezeformer design of the top-performing Google ISLR solution. As shown in Fig. 4.3, a stem convolution projects the 708 input features to a 192-dimensional representation, which passes through **two repetitions of three Conv1D blocks followed by a transformer block**, then global average pooling and a 250-way softmax classifier.

The stem projection maps each 708-dimensional frame to the 192-dimensional working representation:

$$h_t^{(0)} = f_t W_{stem} + b_{stem}, W_{stem} \in \mathbb{R}^{708 \times 192}$$

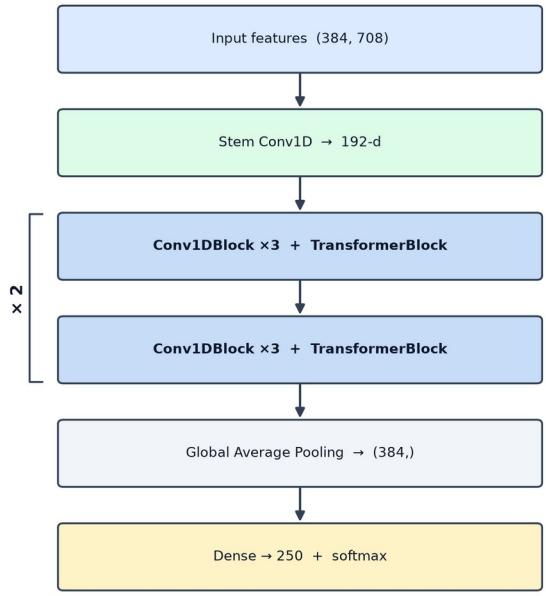

Figure 4.3: ASL macro architecture

Each Conv1D block (Fig. 4.4) combines a point-wise convolution (kernel 1), a **causal depthwise convolution** (kernel 17) with batch normalisation, Efficient Channel Attention, and a second point-wise convolution, wrapped in a residual connection.

Writing PW for a point-wise (kernel-1) convolution and DW for the depthwise convolution, one Conv1D block computes:

$$PW(x) = xW + b,\ swish(x) = x\,\sigma(x),\ \sigma(x) = 1/1 + e^{-x}$$

$$(DW\,x)_{c,t} = \sum_{k=0}^{16} w_{c,k}\,x_{c,t-16+k}\,(causal,\ kernel\ \ 17)$$

$$BN(x) = \gamma\,\frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

$$x' = x + PW_2(ECA(BN(DW(swish(PW_1(x))))))$$

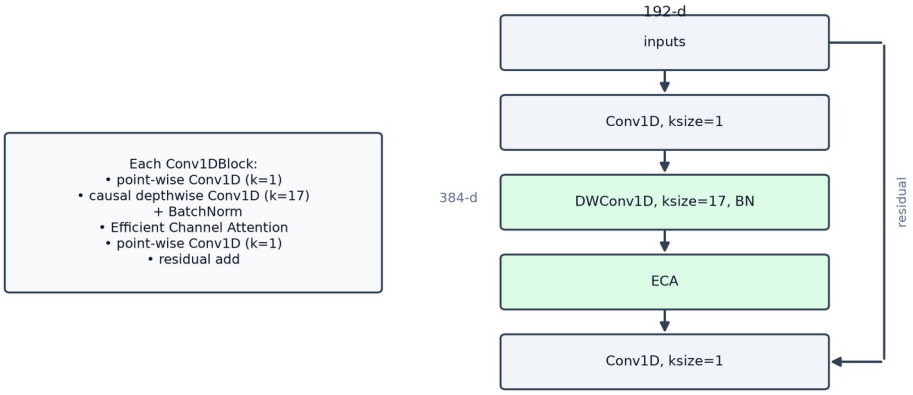

Figure 4.4: Conv1DBlock internals

Efficient Channel Attention (Fig. 4.5) recalibrates channels cheaply: it global-average-pools each channel, applies a small 1-D convolution across channels, and scales the features by the resulting sigmoid weights.

With T the number of frames and sigma the logistic function, Efficient Channel Attention is:

$$z_c = \frac{1}{T}\sum_{t=1}^{T} x_{c,t}, a = \sigma\left(Conv1D_{k=5}(z)\right), \tilde{x}_{c,t} = a_c x_{c,t}$$

Figure 4.5: Efficient Channel Attention

The transformer block (Fig. 4.6) follows the standard design [29]: layer normalisation, multi-head self-attention, a residual connection, then a feed-forward MLP with its own residual.

The attention sub-layer uses scaled dot-product self-attention with H heads of dimension d_k; the feed-forward sub-layer is a two-layer MLP:

$$Attention(Q,K,V)=softmax(\frac{QK^{\top}}{\sqrt{d_k}})V,d_k=48$$

$$MHSA(x)=[head_1\|\cdots\|head_H]W_O,H=4$$

$$FFN(x)=swish(xW_1+b_1)W_2+b_2$$

The deployed TFLite model is a three-tower ensemble: three independently trained towers of this architecture are averaged at the logit level before the final soft-max. Global average pooling, the linear classifier, and the ensemble average are:

$$v=\frac{1}{T}\sum_{t=1}^{T}h_t(GAP),l^{(m)}=vW_{cls}+b,W_{cls}\in\mathbb{R}^{192\times250}$$

$$\acute{l}=\frac{1}{3}\sum_{m=1}^{3}l^{(m)},p=softmax(\acute{l})\in\mathbb{R}^{250}$$

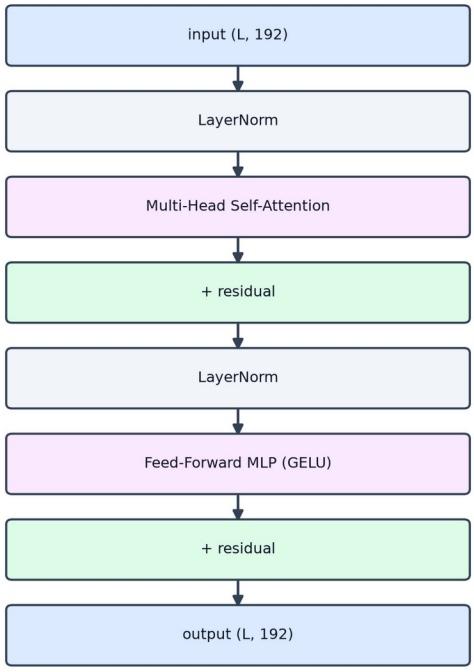

Figure 4.6: TransformerBlock

A key detail is the use of causal padding in the depthwise convolutions (Fig. 4.7): unlike "same" padding, causal padding prevents padded future frames from contaminating the output, preserving the temporal order during streaming inference.

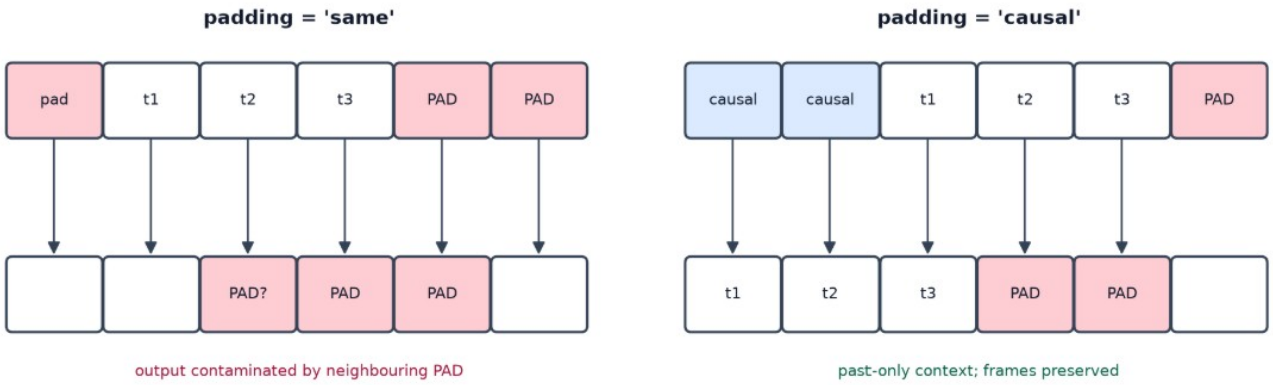

Figure 4.7: Causal vs. "same" padding

4.2.3 Data augmentation

To generalise across signing speeds, camera angles, and occlusions, the ASL model is trained with the dual-modality augmentation of Fig. 4.8: temporal resampling (0.5×–1.5×), temporal masking (20–40% of frames blanked), horizontal flipping (for left-handed signers), random affine transforms (scale, shift, shear, rotation up to ±30°), and spatial cutout.

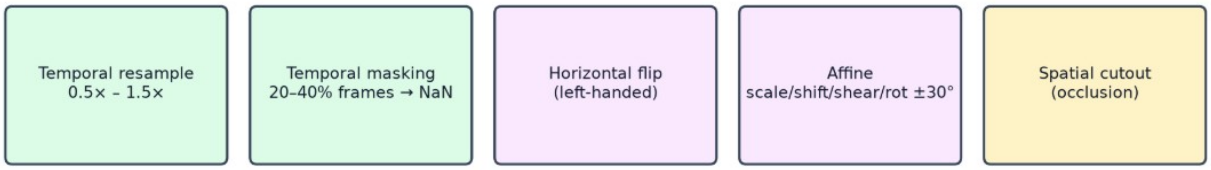

Figure 4.8: ASL data augmentation.

4.2.4 Training and results

The training configuration is summarised in Fig. 4.9: the RAdam optimizer with a Lookahead wrapper, a base learning rate of 5e-4 scaled to 4e-3 across replicas under a cosine decay (no warmup), and a categorical cross-entropy loss with label smoothing of 0.1, over 400 epochs. Three regularizers combat overfitting on the 250 classes: Drop-Path (stochastic depth, p = 0.2), late dropout (p = 0.8 on the final dense layer), and Adversarial Weight Perturbation (λ = 0.2).

The training objective is categorical cross-entropy with label smoothing (epsilon = 0.1) over K = 250 classes:

$$\tilde{y}_c = (1-\varepsilon)\, y_c + \frac{\varepsilon}{K}, \varepsilon = 0.1, K = 250$$

$$L_{CCE} = -\sum_{c=1}^{K} \tilde{y}_c \log p_c$$

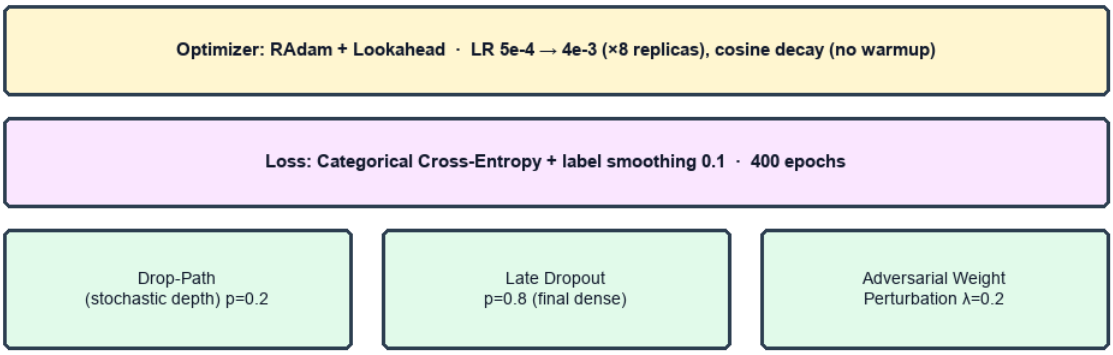

Figure 4.9: ASL training configuration & regularization

The resulting training curve is shown in Fig. 4.10. The model reaches a validation accuracy of approximately 0.80. Validation accuracy exceeding training accuracy is expected here, because the heavy augmentation and regularization are active only during training.

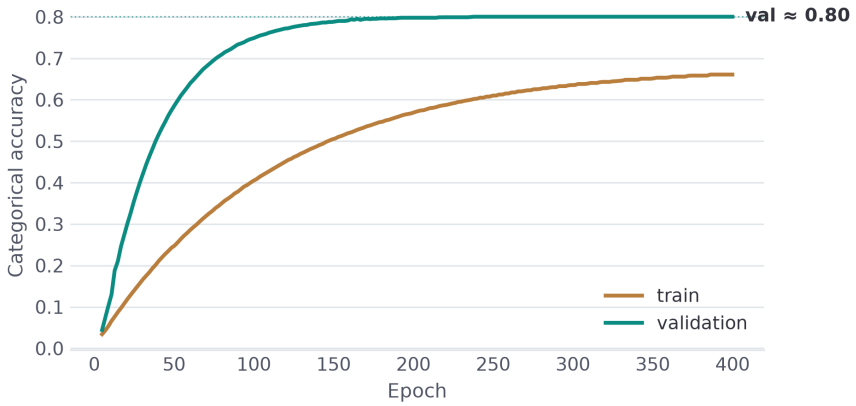

Figure 4.10a: Training and validation accuracies

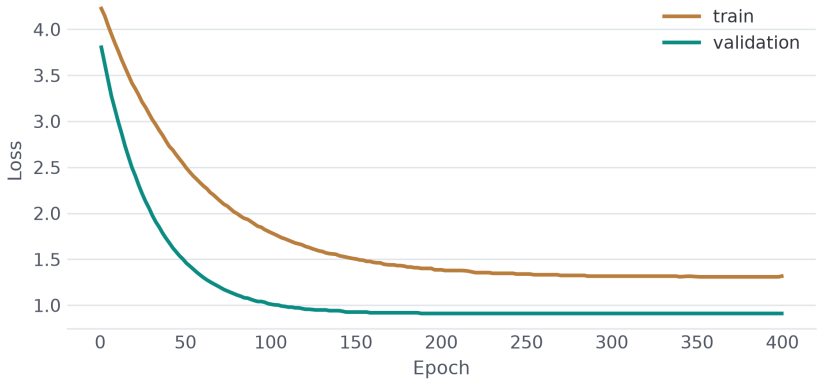

Figure 4.10b: Training and validation loss

4.3 The ArSL Recognition model

4.3.1 Preprocessing

The ArSL pipeline (Fig. 4.11) extracts **59 MediaPipe landmarks** — 17 upper-body pose points (indices 0–16) and 21 points per hand — from each clip. The z axis is **zeroed** (only x, y carry information) to avoid depth-scaling distortion across phone cameras; coordinates are centred on the shoulder midpoint and scaled by the shoulder width for position and scale invariance; and every clip is resampled to a fixed **30 frames**, producing a (30, 177) tensor.

With p_L, p_R the shoulder landmarks (indices 11, 12) and T the captured frame count, the normalization and resampling are:

$$z \leftarrow 0, c = 1/2(p_L + p_R), s = \| p_L - p_R \|_2$$

$$\hat{p}_{t,i} = \frac{p_{t,i} - c}{s}, \tau_k = \left\lfloor \frac{k(T-1)}{29} \right\rfloor, k = 0, \ldots, 29$$

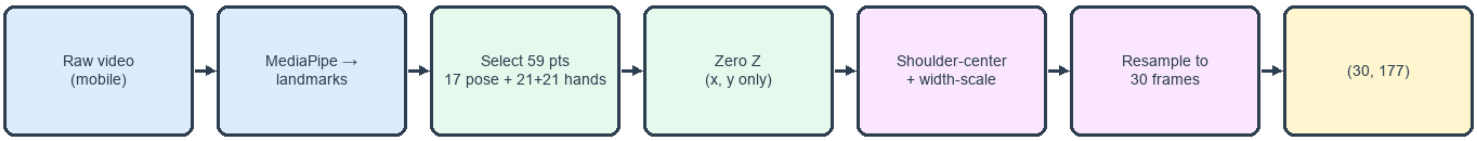

Figure 4.11: ArSL model

4.3.2 Architecture

The ArSL model is a compact CNN-GRU (Fig. 4.12). Two 1-D convolutional blocks ($177 \rightarrow 128$ then $128 \rightarrow 128$, kernel 3, each with batch normalisation and ReLU) extract local spatio-temporal features; a two-layer **bidirectional GRU** [30] ($128 \rightarrow 64$ hidden units, dropout 0.3) models the temporal dynamics; and two fully connected layers ($128 \rightarrow 64$ with ReLU and dropout 0.5, then $64 \rightarrow 20$) with a softmax produce the prediction.

Each convolutional block applies batch normalization and ReLU; at inference the soft-max outputs are averaged over three temporal windows before thresholding:

$$h = ReLU\left(BN\left(W * x + b\right)\right), ReLU\left(x\right) = max\left(0, x\right)$$

$$o = softmax\left(W_2\, drop\left(ReLU\left(W_1 g\right)\right) + b_2\right) \in \mathbb{R}^{20}$$

$$\acute{p} = \frac{1}{|W|} \sum_{w \in W} softmax\left(z^{(w)}\right), W = \{30, 45, 60\}$$

$$\hat{y} = \arg\max_i \acute{p}_i, accept \quad iff \quad \acute{p}_{\hat{y}} > \tau, \tau = 0.45$$

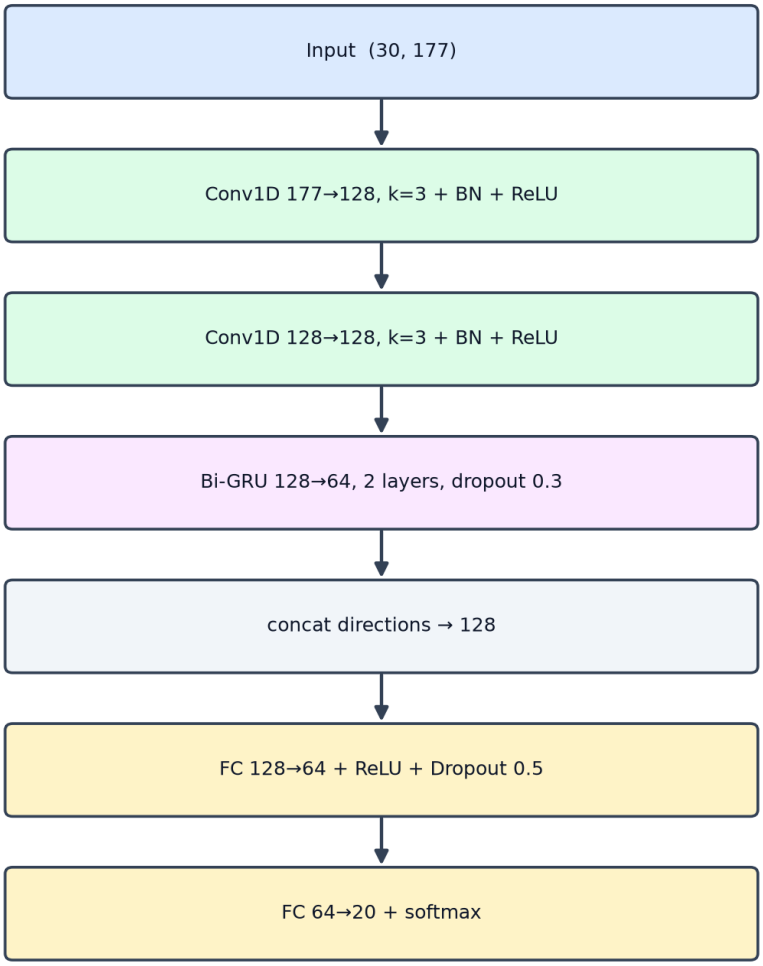

Figure 4.12: ArSL CNN-GRU architecture

The bidirectional GRU (Fig. 4.13) processes the 30-frame sequence in both temporal directions and concatenates the forward and backward hidden states, so each prediction is informed by the full context of the sign.

Each GRU layer updates its hidden state through reset and update gates, and the bidirectional layer concatenates the two directions:

$$r_t = \sigma\left(W_{ir} x_t + W_{hr} h_{t-1} + b_r\right), z_t = \sigma\left(W_{iz} x_t + W_{hz} h_{t-1} + b_z\right)$$

$$n_t = \tanh\left(W_{\text{in}} x_t + r_t \odot \left(W_{hn} h_{t-1}\right) + b_n\right), h_t = \left(1 - z_t\right) \odot n_t + z_t \odot h_{t-1}$$

$$\overleftrightarrow{h}_t = [\vec{h}_t \| \acute{h}_t]$$

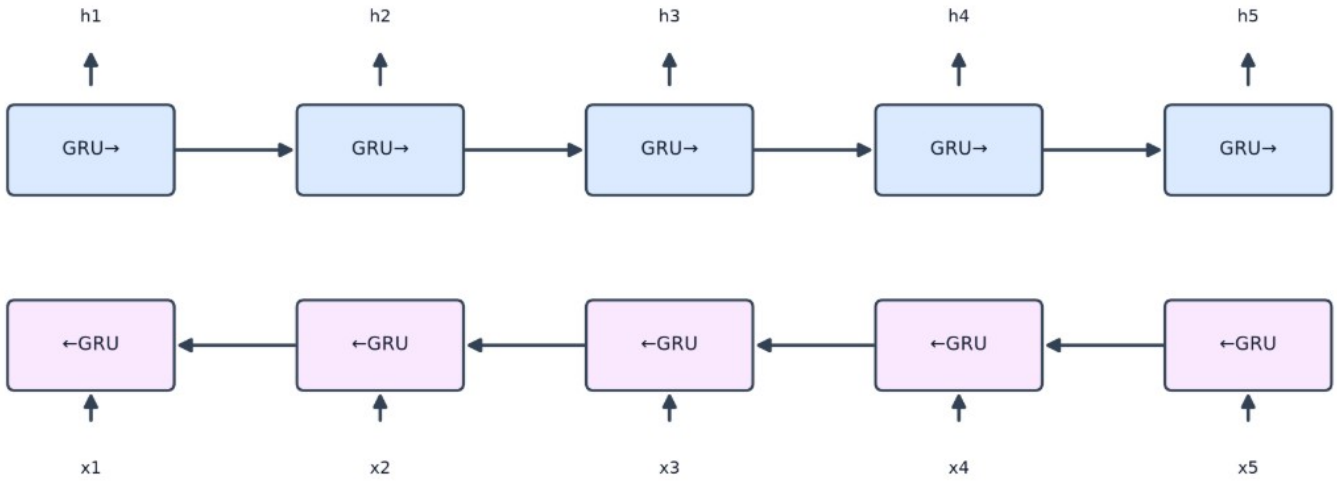

Figure 4.13: Bidirectional GRU (unfolded)

4.3.3 Data augmentation

The ArSL model is trained with the four online augmentations of Fig. 4.14: horizontal mirroring (50%), inactive-hand masking (70%, detected by spatial variance), affine scaling (0.92×–1.08×) with Gaussian noise (σ = 0.008), and temporal jittering.

Figure 4.14: ArSL data augmentation

4.3.4 Training and results

The training configuration is summarised in Fig. 4.15: the Adam optimizer (weight decay 1e-4) at a learning rate of 1e-3 with a ReduceLROnPlateau schedule (halving after three stagnant epochs), a standard cross-entropy loss, and a batch size of 64, for up to 100 epochs with early stopping (patience 12). Regularisation is provided by dropout in the GRU (0.3) and the dense layer (0.5) together with the augmentation above.

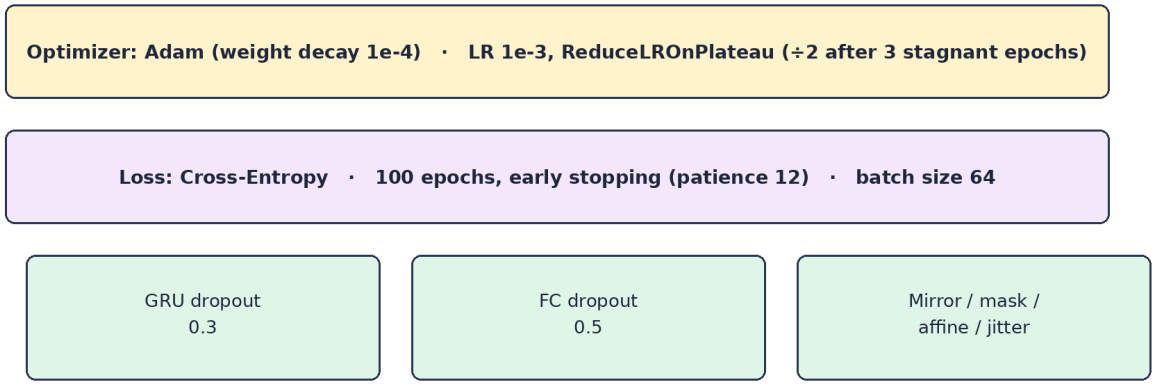

Figure 4.15: ArSL training configuration

The resulting training curves are shown in Fig. 4.16. The model converges quickly on the 20-class task and attains **99.41%** accuracy on its test split.

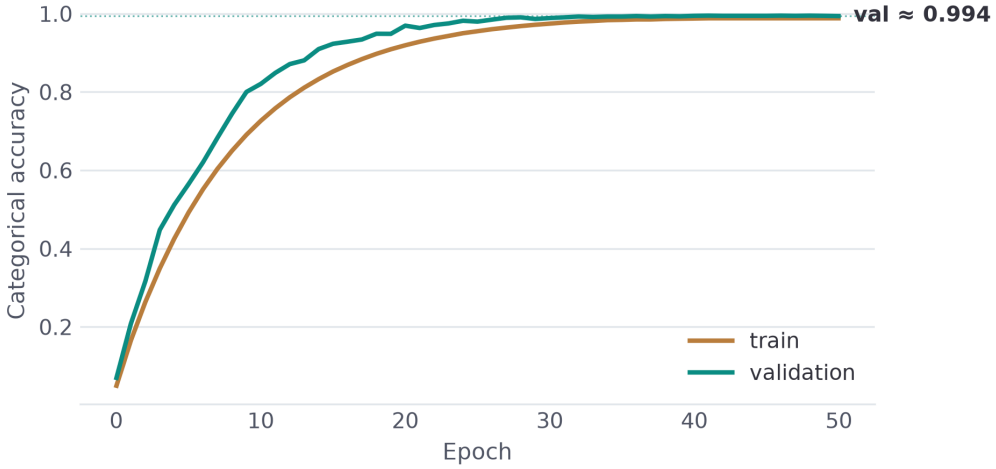

Figure 4.16a: Training and validation accuracies

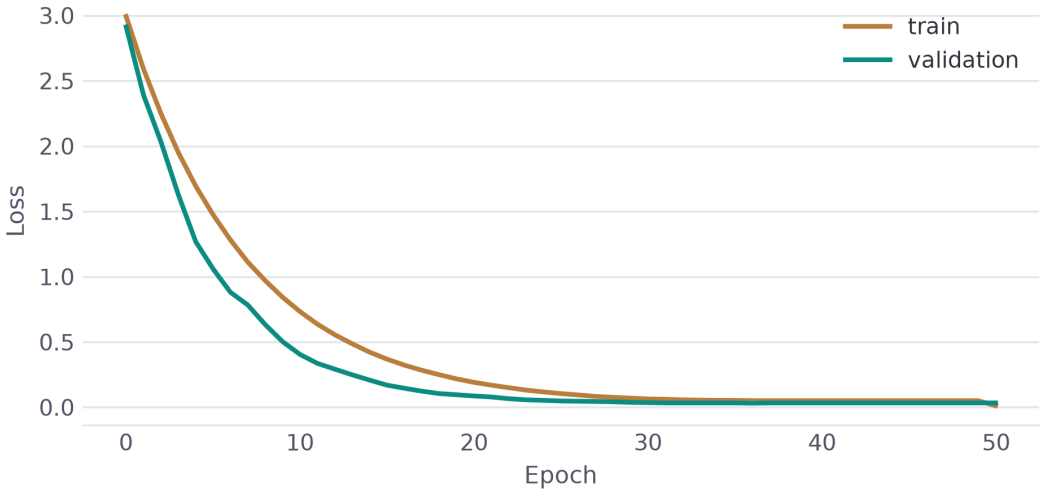

Figure 4.16b: Training and validation loss

4.4 Gloss-to-Sentence Translation

Once a gloss sequence has been committed, it is converted into a fluent sentence by a large language model behind a single endpoint. The primary provider is Gemini [27], prompted with few-shot examples that enforce Topic–Comment reordering and the insertion of function words; results are cached for repeated phrases. When no cloud provider is reachable, the chain falls back to a local LLaMA-based model served by Ollama [28], and ultimately to emitting the raw gloss, so the feature degrades gracefully rather than failing.

4.5 Sign Synthesis

The reverse direction maps a sentence to gloss and retrieves a sign for each token. Rather than exact string matching, retrieval uses **Sentence-BERT** embeddings [26] stored in pgvector, so a synonym or paraphrase still selects the correct sign. The retrieved landmark sequences are stitched and played by the avatar.

4.6 Real-Time Integration

4.6.1 ASL model

The runtime pipeline is shown in Fig. 4.17. In the browser, webcam frames at 30 FPS are converted to landmarks, smoothed and gap-filled (missing hands sent as NaN, not zero), and buffered to 60 frames before being posted to /api/translate. The server runs the TFLite interpreter, applies the 0.80 acceptance gate, performs a majority vote over the last 15 predictions, accumulates gloss, and — after five seconds of no hands — calls the language model and /api/tts. The meeting mode layers this on a peer-to-peer WebRTC connection [31] coordinated by Socket.IO.

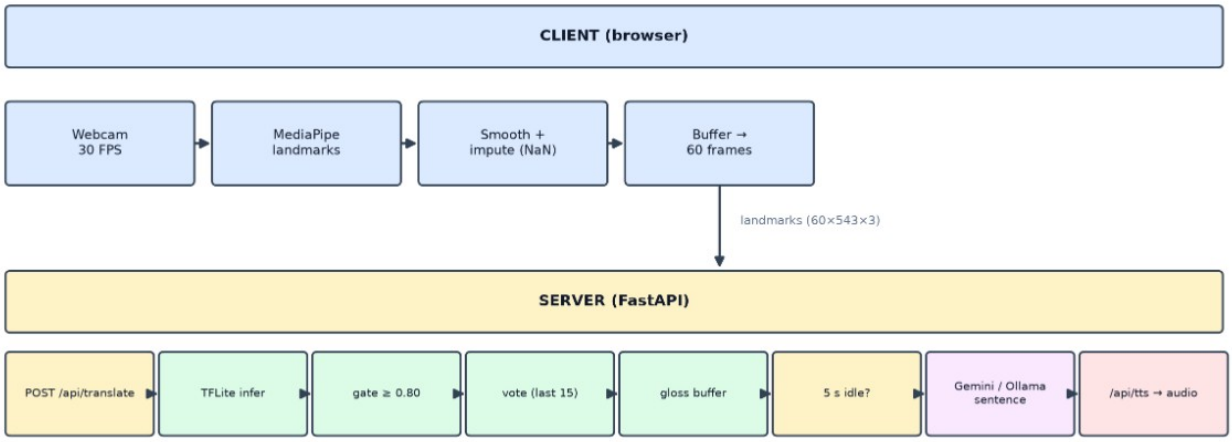

Figure 4.17: ASL Runtime inference & integration pipeline

4.6.2 ArSL model

The ArSL model follows an analogous pipeline, shown in Fig. 4.18, with two differences: its preprocessing runs **server-side in PyTorch** rather than inside a TFLite graph — resampling the buffered clip to 30 frames over the 59 selected landmarks — and its acceptance gate is **0.45** rather than 0.80.

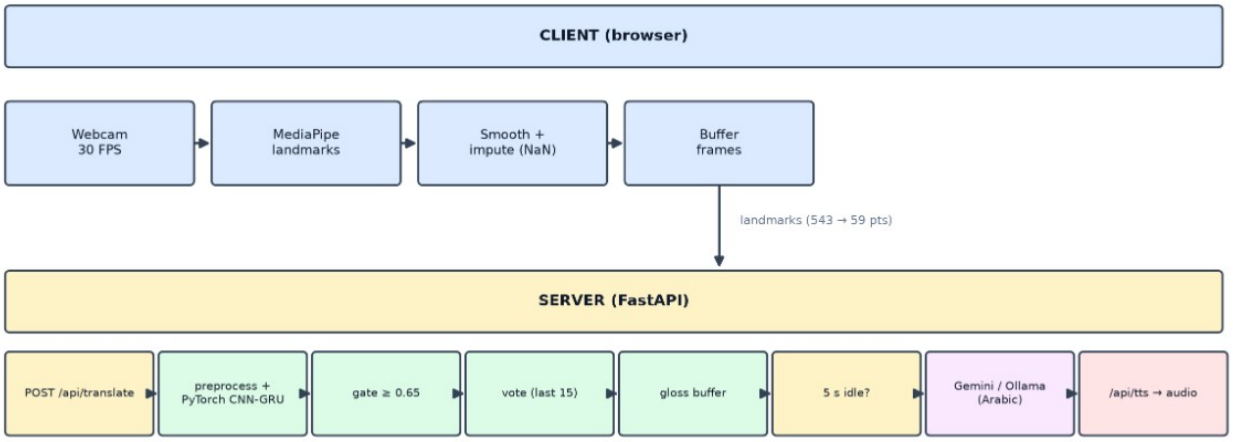

Figure 4.18: ArSL runtime inference & integration pipeline

4.7 Model Comparison

Table 4.1: ASL vs. ArSL model comparison.

Attribute	ASL (English Model)	ArSL (Arabic Model)
Dataset	Google ISLR (250 classes)	Balaha ArSL (20 classes)
Input Shape	raw 60 × 543 × 3 → 708 features	30 × 177 (59 keypoints)
Z-Coordinate Used	No (x, y only)	No (explicitly zeroed to 0.0)
Backbone Architecture	1D-CNN + Squeezeformer / Transformer	CNN + Bi-GRU
Export Format	TFLite	PyTorch (.pth)
Loss Function	CCE with label smoothing (0.1)	Cross-Entropy Loss
Model Accuracy	80.00%	99.41%

4.8 Comparison with Prior Recognition Architectures

Isolated sign-language recognition has moved through several architectural generations, and Together's two models sit at different points along that path. The earliest deep recognisers were appearance-based: a two-dimensional convolutional neural network (CNN) extracted per-frame features from raw video, which a recurrent network — usually a long short-term memory (LSTM) or its bidirectional variant (Bi-LSTM) — then aggregated over time. Three-dimensional CNNs such as I3D folded space and time into a single convolution and pushed accuracy higher on large benchmarks, but at a steep computational cost and with a heavy dependence on video data [6].

Because raw video is expensive and privacy-invasive, the field increasingly favours pose-based models that operate on extracted skeleton keypoints rather than pixels. Recurrent models (RNN, LSTM, Bi-LSTM) remain a natural fit for these keypoint sequences and are still common in Arabic Sign Language (ArSL) work, where datasets are small. Hybrid CNN-LSTM and CNN-GRU networks improve on pure recurrent models by letting a convolutional front end learn local

spatio-temporal patterns before the recurrent layer models longer-range order; this compact design is well suited to low-resource vocabularies. More recently, graph convolutional networks (GCNs) that treat the skeleton as a graph have won signer-independent challenges with around 98% accuracy [9], and attention-based Transformers — including the convolution-augmented Squeezeformer — now lead the large-vocabulary Google Isolated Sign Language Recognition benchmark [23].

Together deliberately chooses a different architecture for each language to match its data budget. The ArSL recogniser is a compact CNN-Bi-GRU: with only 8,437 samples across 20 signs, a small convolutional front end followed by a bidirectional GRU captures the temporal dynamics of a sign without the parameter count — and the overfitting risk — of a Transformer. The ASL recogniser instead adopts the Squeezeformer-style Conv1D-plus-Transformer design that dominates the 250-class Google ISLR task, trading a larger parameter budget for the long-range modelling that a 250-way vocabulary demands, and is exported as a three-tower ensemble for additional robustness. In short, the two models bracket the architectural spectrum reviewed above — a lightweight recurrent hybrid for the low-resource language and an attention-based convolutional network for the high-resource one — rather than forcing a single design onto two very different datasets. A side-by-side summary is given in Table 4.2.

Table 4.2: Neural architecture families for sign-language recognition and where Together's two models sit.

Architecture	Input	How it models a sign	Trade-offs	Representative sign-language work
2D-CNN + RNN/LSTM	RGB frames	Per-frame CNN features aggregated by a recurrent net	Strong on appearance; compute-heavy; background-sensitive	WLASL appearance baselines [6]
3D-CNN (e.g. I3D)	RGB clips	Joint spatio-temporal convolution over the clip	Captures motion directly; very data- and compute-hungry	I3D on WLASL [6]
RNN / LSTM	Keypoints	Sequential hidden state over time	Lightweight; weak on long-range context	LSTM recognisers surveyed in [3]
Bi-LSTM	Keypoints	Reads the sequence in both directions	Full temporal context; ~2x cost; still sequential	Bi-LSTM recognisers surveyed in [3]

CNN-LSTM / CNN-GRU	Keypoints	CNN learns local features, RNN models time	Compact, data-efficient; ideal for small vocab	Together — ArSL model
GCN (skeleton)	Pose graph	Graph convolutions over skeleton joints	Parameter-efficient; signer-independent; needs clean keypoints	AUTSL winner ~98% [9]
Transformer / Squeezeformer	Keypoints	Self-attention (+ Conv1D) over all frames	Long-range, parallel; data-hungry	Together — ASL model; Google ISLR [23]

4.9 The Together Application: Modules and User Interface

While the preceding sections described the system's internals, the user interacts with Together as a single bilingual progressive web application (PWA) made up of eight modules. Five are translation modules built on a shared real-time dashboard, and three — Dictionary, Practice, and Analytics — support reference, learning, and self-monitoring. Every view is fully bilingual: it mirrors between English (left-to-right) and Arabic (right-to-left), offers a dark and light theme, and the application is installable and responsive across desktop and mobile.

4.9.1 Translation modules and the shared dashboard

The five translation modules expose the directional pipelines of Sections 4.2–4.6, each branded and tuned for one task (Table 4.3). HandScript and VoiceBridge consume the sign-recognition path, rendering recognised signs as on-screen text and as synthesised speech respectively; SignType and TalkSide drive the reverse, sign-synthesis path from typed text and from transcribed speech; and SignLine combines both directions in the live two-party meeting of Section 4.6.

Table 4.3: The five translation modules and the pipelines they expose.

Module	Direction	Pipeline used	Output
HandScript	Sign → Text	Recognition → gloss → LLM	On-screen sentence
VoiceBridge	Sign → Speech	Recognition → gloss → LLM → TTS	Synthesised speech
SignType	Text → Sign	Text → gloss → avatar	Signing avatar

TalkSide	Speech → Sign	STT → gloss → avatar	Signing avatar
SignLine	Sign ⇄ Speech (live)	Both pipelines over WebRTC	Live captions + avatar

All four single-user translation modules share one real-time dashboard (Fig. 4.19). A header row of status cards reports the signs recognised today, the running average confidence, the elapsed session time, and the active module with its state (idle, listening, or inferring). The Live Camera panel hosts the webcam preview and a Start/Stop control; accepted signs stream into a Session Log, while a compact Analytics side-panel shows a live confidence ring, signs per minute, and session progress. Below the camera, a Transcription panel accumulates the recognised gloss and turns it into a grammatical sentence — automatically, or on demand through an Edit & Compose control; pressing Enter invokes the language-model stage of Section 4.4.

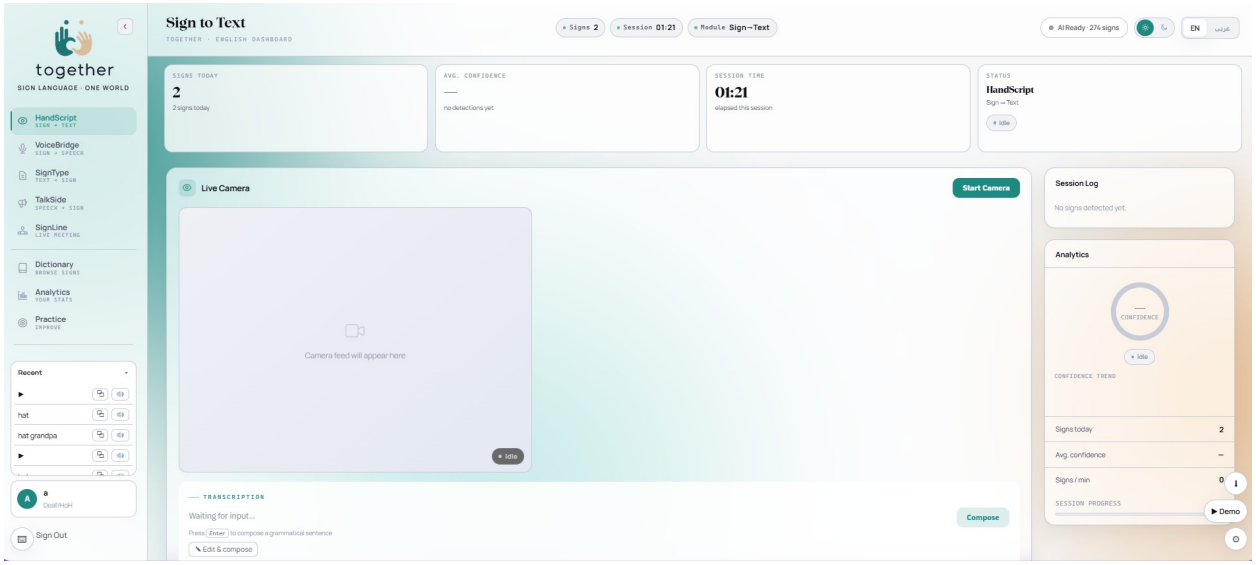

Figure 4.19: The shared real-time translation dashboard (HandScript, sign-to-text).

4.9.2 Dictionary

The Dictionary module turns the sign database of Section 4.5 into a browsable reference. Signs can be filtered by category or located through a search box backed by the Sentence-BERT semantic lookup, so a synonym or paraphrase still resolves to the correct entry. Each result is

shown in two complementary forms: the landmark-driven avatar that renders the sign, and — optionally — the original video recorded by a human signer, letting learners compare the synthetic animation against authentic articulation.

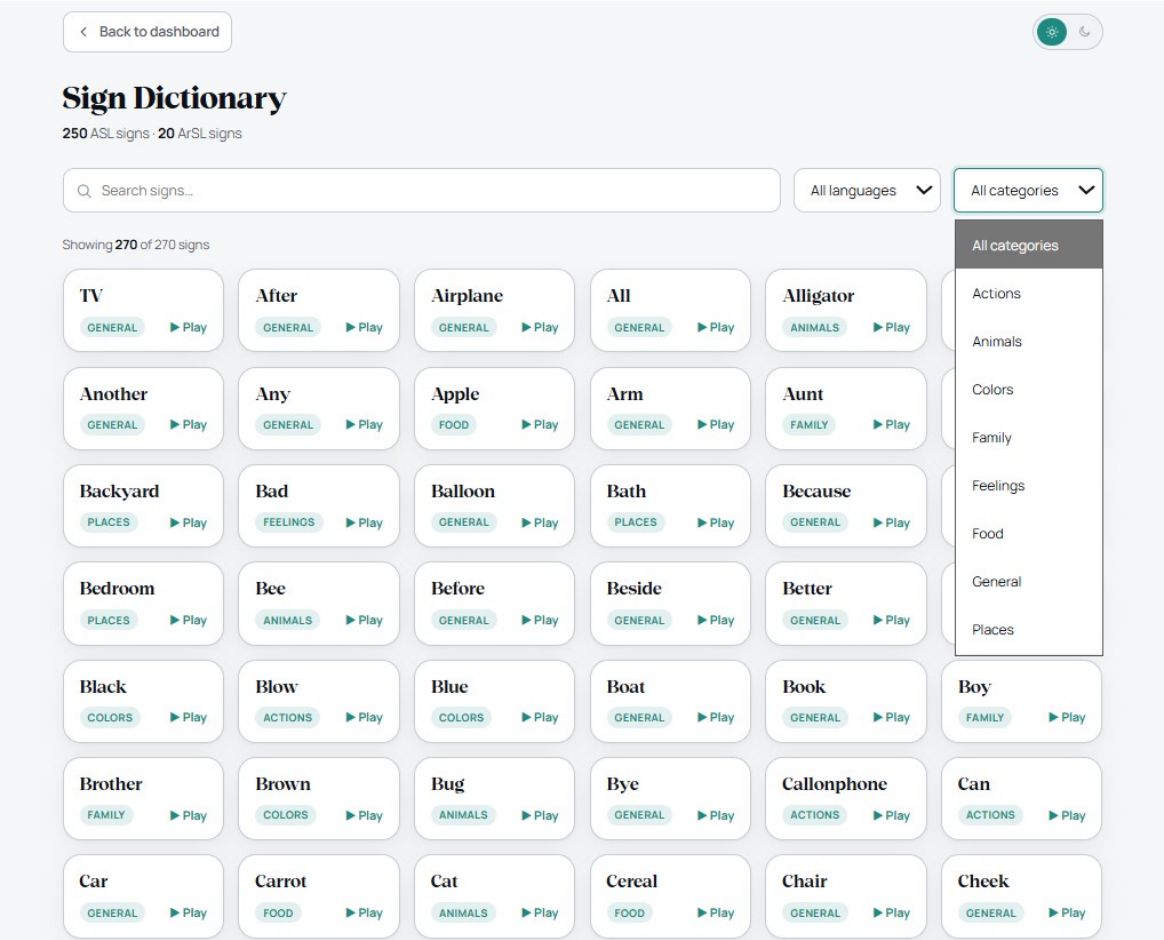

Figure 4.20: Dictionary: category browsing and search, with avatar and original-signer video.

4.9.3 Analytics

The Analytics page (Fig. 4.21) summarises a user's activity entirely on-device, consistent with the privacy stance of NFR8 — only landmarks, never raw video, leave the browser. It reports total detections, unique signs, average confidence, and signing speed (signs per minute), and visualises a confidence trend across the session alongside the most-detected signs and a

breakdown by language (ASL / ArSL) and input module (Vision / Speech). A session can be exported as CSV or printed to PDF for record-keeping.

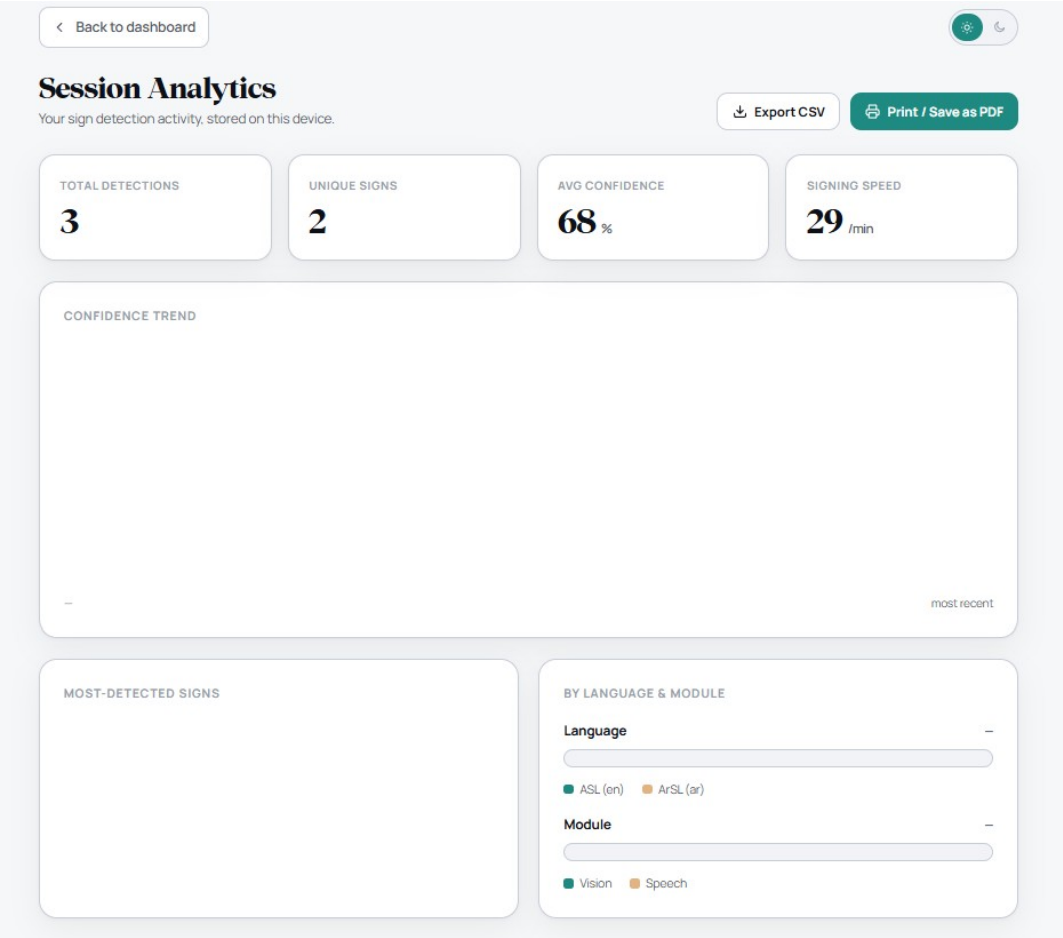

Figure 4.21: Session Analytics: on-device detection statistics, confidence trend, and language/module breakdown.

4.9.4 Practice

The Practice module is a guided drill for learners. Each round presents ten signs drawn at random from the selected language (ASL or ArSL), each demonstrated by the avatar; after watching a prompt, the learner attempts the sign to the webcam and the recognition pipeline judges whether it was produced correctly, giving immediate feedback before advancing. Because

Practice reuses the same recognition models and confidence gating as the live translation modules, a learner is measured against exactly the system that powers real translation.

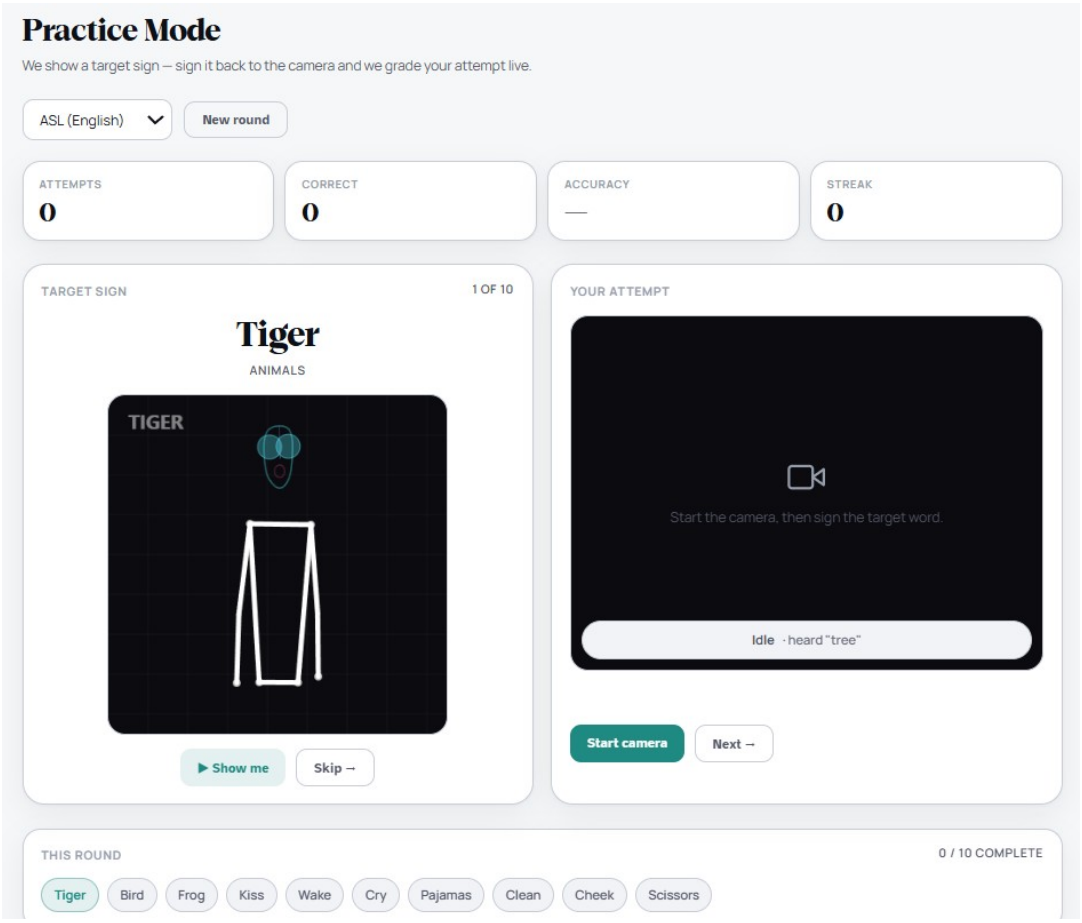

Figure 4.22: Practice: avatar-prompted rounds of ten signs with live attempt feedback.

4.9.5 Bilingual, responsive interface

Every module is delivered through one design system in two fully mirrored locales. The Arabic interface is fully translated into Arabic — every label, message, and control, not merely the font — and is presented as a true right-to-left mirror of the layout, so the text, navigation, charts, and avatar all read and reflect correctly with no English left in the interface. The same views adapt from desktop to mobile, where the application can be installed as a PWA and used offline for the

components that do not depend on a cloud provider. Figures 4.23 and 4.24 show representative desktop and mobile screens, including the right-to-left Arabic layout.

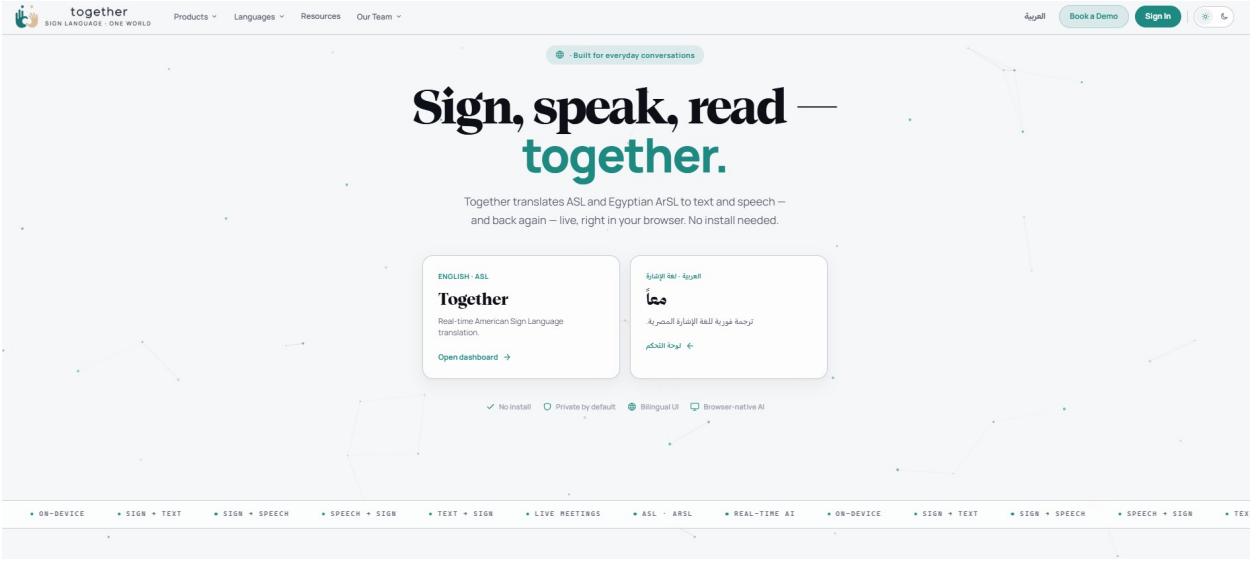

Figure 4.23: Representative desktop screen (English, left-to-right).

Figure 4.24: Representative mobile screen, including the right-to-left Arabic interface.

4.10 Authentication and Security

Access to the platform is protected by a token-based authentication layer. Users register and log in with a password that is never stored in plaintext: credentials are hashed with Argon2id, a memory-hard algorithm resistant to brute-force and GPU-accelerated attacks, and only the resulting hash is persisted (Section 3.3.2). On a successful login the server issues a short-lived JSON Web Token (JWT) access token together with a longer-lived refresh token; refresh tokens are themselves stored only as hashes, expire, and are rotated on each use, so a leaked or stale refresh token cannot be replayed indefinitely.

Account creation is further verified through a WhatsApp-delivered one-time password (OTP). During registration, the platform sends an OTP from its dedicated WhatsApp account to the user’s WhatsApp number, and the account is activated only once the code is entered correctly, tying every account to a real, reachable phone number. Delivering the code over WhatsApp rather than SMS requires no additional telephony infrastructure and reaches users on a channel they already use, while blunting bot sign-ups and throw-away registrations.

Every authenticated and recognition endpoint requires a valid access token, and incoming requests are bounded by a sliding-window rate limiter that blunts credential-stuffing and abuse; standard security headers and a restrictive cross-origin (CORS) policy further harden the API. Combined with the privacy design of NFR8 — only landmarks, never raw video, leave the browser — this ensures that both a user's account and their captured signing data remain protected end to end. The full authentication flow (registration, login, token issue and refresh, and rate limiting) is exercised by the automated test suite of Section 5.5.

Chapter 5: Testing, Validation and Results

This chapter evaluates *Together* comprehensively. It defines the evaluation methodology, reports the recognition results for both models (including generalization, per-class metrics, and confusion analysis), measures the quality of the gloss-to-sentence translation against human references, assesses system performance (latency and model quantization), documents the functional and system testing of every part of the platform, and closes with the limitations and threats to validity.

5.1 Evaluation Methodology

Four families of evidence are used.

- **Recognition** is measured by classification **accuracy**, by per-class **precision, recall, and F1**, and by **confusion matrices**. To test generalization rather than only memorisation, each model is additionally evaluated on data drawn from outside its training distribution: the ASL model is tested **cross-dataset** on the independent SignASL benchmark, and the ArSL model is tested **signer-independently** (held-out signers).
- **Translation quality** — the contribution of the language-model stage — is measured against human reference sentences with the **BLEU** [32] and **chrF** [33] metrics, computed with a standardised, reproducible scorer [34], and always compared against a *raw-gloss baseline* so the language model's contribution is isolated rather than asserted. A held-out set of 20 reference sentences per language is used.
- **System performance** is assessed by end-to-end and per-stage **latency** and by the effect of **model quantization** on size and speed.
- **Functional correctness** is assessed by an automated test suite, a frontend regression module, and manual end-to-end testing of every site module.

All metrics used in this chapter are defined in Table 5.1.

Table 5.1: Evaluation metrics used in this chapter.

Metric	Description	Expression
Accuracy (ACC)	Correct predictions over the total number of predictions.	$\frac{TP+TN}{TP+TN+FP+FN}$
Top-1 accuracy	Share of cases where the highest-confidence prediction matches the true label.	$\frac{\text{correct (top-1)}}{\text{total samples}}$
Top-5 accuracy	Share of cases where the true label is among the model's top five predictions.	$\frac{\text{true label in top 5}}{\text{total samples}}$
Precision (P)	Correct positives over all predicted positives.	$\frac{TP}{TP+FP}$
Recall (R)	Correct positives over all actual positives.	$\frac{TP}{TP+FN}$
F1 score	Harmonic mean of precision and recall; range $[0,1]$.	$\frac{2TP}{2TP+FP+FN}$
Confusion matrix	Cross-tabulates true vs. predicted classes; entry C_{ij} counts true class i predicted as j.	Cij=#{true=i, pred=j}
Categorical cross-entropy (loss)	Multi-class training loss, here with label smoothing ($\varepsilon = 0.1$).	$-\sum_{i=1}^{C} \tilde{y}_i \log\ p_i$
BLEU	Translation: word n-gram precision with a brevity penalty (BP).	$BP \cdot \exp!\left(\sum_{n=1}^{N} w_n \ln\ p_n\right)$
chrF	Translation: character n-gram F-score (recall weighted by β).	$(1+\beta^2)\frac{chrP \cdot chrR}{\beta^2 chrP + chrR}$

5.2 Recognition Results

5.2.1 Accuracy and generalization

The accuracy results are summarised in Table 5.2 and Fig. 5.1. On its own test distribution, the **ASL model reaches 80%** accuracy over the 250-class vocabulary. To probe generalization, the same model was evaluated **cross-dataset** on the independent **SignASL** benchmark, where it attains **62.4% Top-1** — a substantial but expected drop, since SignASL is a separate corpus with different signers, recording conditions, and label statistics. That the model still recognises the majority of signs on an unseen corpus is strong evidence that it has learned genuine sign representations rather than dataset-specific artefacts.

The **ArSL model reaches 99.41%** accuracy on its in-distribution (stratified 70/15/15) test split. Because that split shares signers across partitions, the model was additionally evaluated under a **signer-independent** protocol — testing on signers unseen during training — where it attains **88%**. The roughly eleven-point gap quantifies how much of the in-distribution score depends on signer-specific cues, and an 88% signer-independent result remains a strong outcome for a 20-class recogniser on phone-camera video.

Table 5.2: Recognition accuracy: in-distribution versus generalization.

Language	Classes	Protocol	Accuracy
ASL	250	In-distribution	80%
ASL	250	Cross-dataset generalization (SignASL)	**62.4%**
ArSL	20	In-distribution	99.41%
ArSL	20	Signer-independent	**88%**

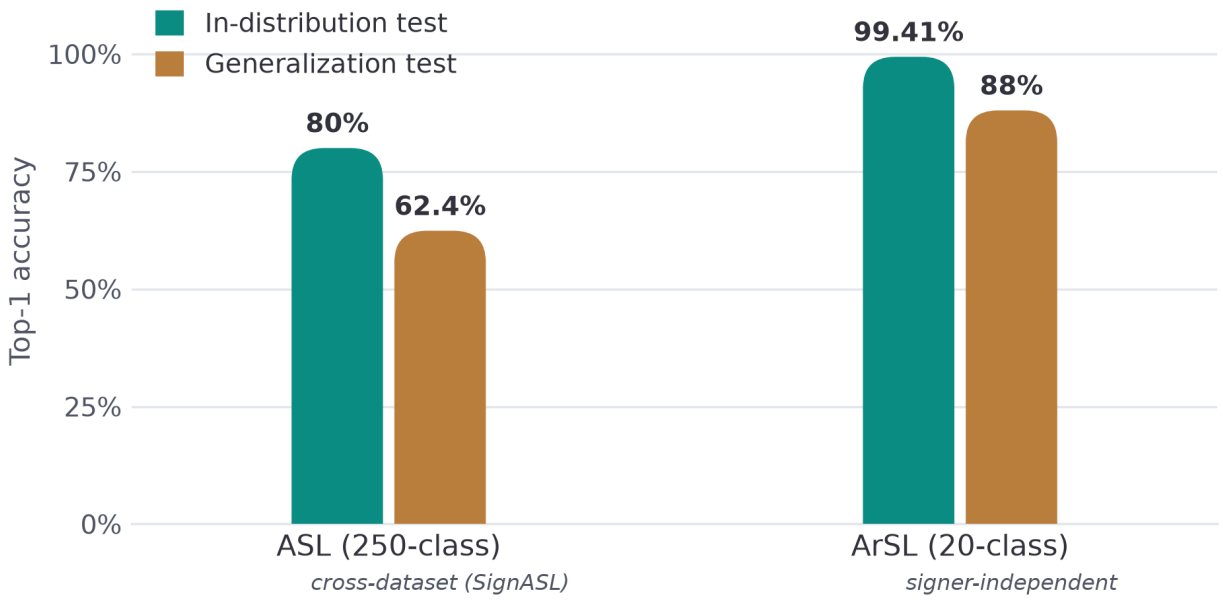

Figure 5.1: Recognition accuracy: in-distribution vs. generalization.

5.2.2 ASL model evaluation metrics

Table 5.3: ASL model evaluation metrics.

Success Metric	**ASL**
Top-1 Accuracy	80.00 %
Top-5 Accuracy	95.00 %
Categorical Cross-Entropy (CCE) Loss	0.90
Macro Precision	0.81
Macro Recall	0.80
Macro F1-Score	0.80

5.2.3 ArSL model evaluation metrics

Table 5.4: ArSL model evaluation metrics.

Success Metric	**ArSL**
Top-1 Accuracy	99.41%
Top-5 Accuracy	100.00%
Categorical Cross-Entropy (CCE) Loss	0.03
Macro Precision	0.99
Macro Recall	0.99
Macro F1-Score	0.99

5.2.4 Confusion analysis

The confusion matrices reveal which signs are mistaken for one another. The ArSL matrix (Fig. 5.2) is small enough to present in full as a 20×20 grid; for the 250-class ASL model a full grid is unreadable, so the most-confused sign pairs are reported instead (Fig. 5.3). Visually similar signs — those differing only in a subtle handshape or movement — are expected to account for most of the off-diagonal mass.

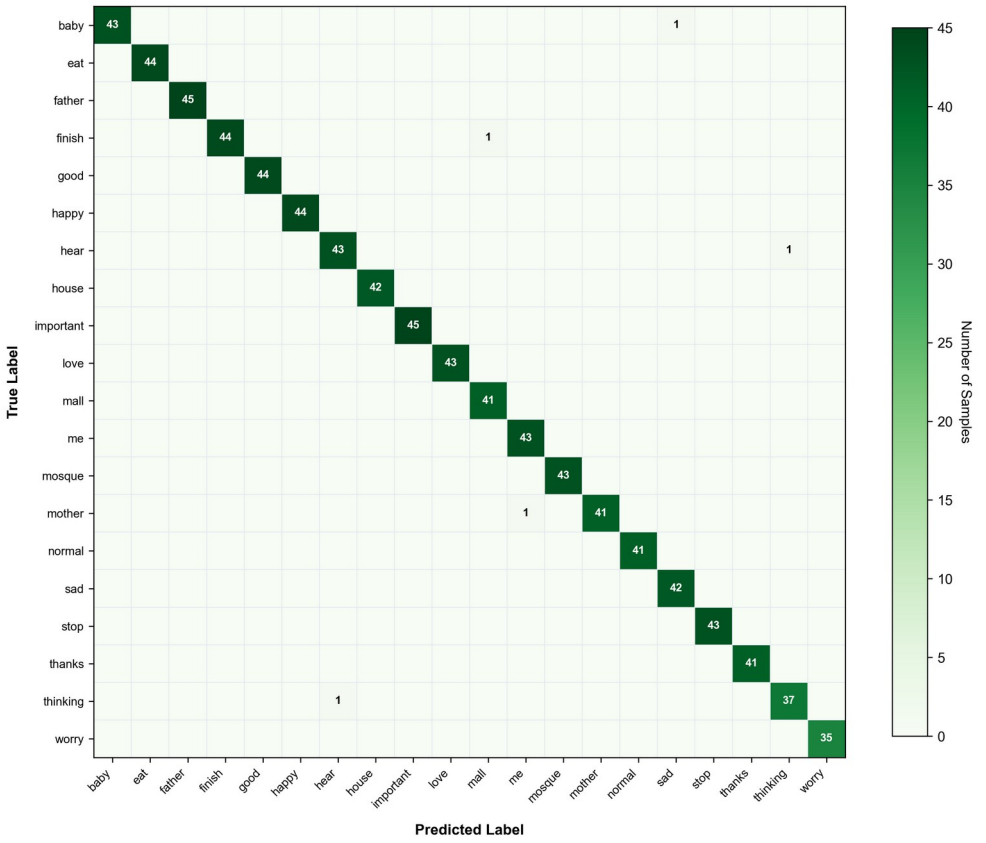

Figure 5.2: ArSL 20×20 confusion matrix

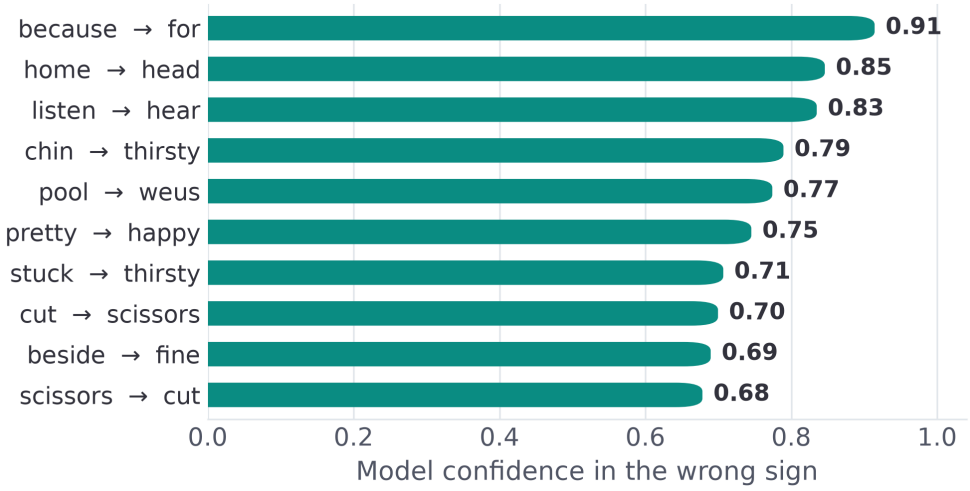

Figure 5.3: ASL confusion analysis (most-confused sign pairs)

5.3 Translation Quality

The translation stage converts the recognised gloss into a fluent sentence; its quality is measured against human reference sentences with BLEU and chrF (Table 5.1), and is always compared against a raw-gloss baseline so that the language model's contribution is isolated rather than asserted.

5.3.1 Results

Table 5.5 reports the scores for both languages, with and without the language model; Figures 5.4 and 5.5 visualise them. In every case the language model produces a large, consistent improvement over the raw-gloss baseline. Both languages are scored on a held-out set of 20 reference sentences; with a set this small, BLEU-4 in particular is high-variance, so chrF and the human-reference comparison carry more weight than the absolute BLEU figures.

Table 5.5: Translation quality (gloss → sentence): LLM vs. raw gloss

Language	System	BLEU-1	BLEU-2	BLEU-3	BLEU-4	chrF
ASL	With LLM	0.945	0.905	0.855	0.805	0.911
ASL	Raw gloss	0.451	0.221	0.091	0.040	0.543
ArSL	With LLM	0.527	0.444	0.424	0.419	0.593
ArSL	Raw gloss	0.211	0.061	0.024	0.026	0.344

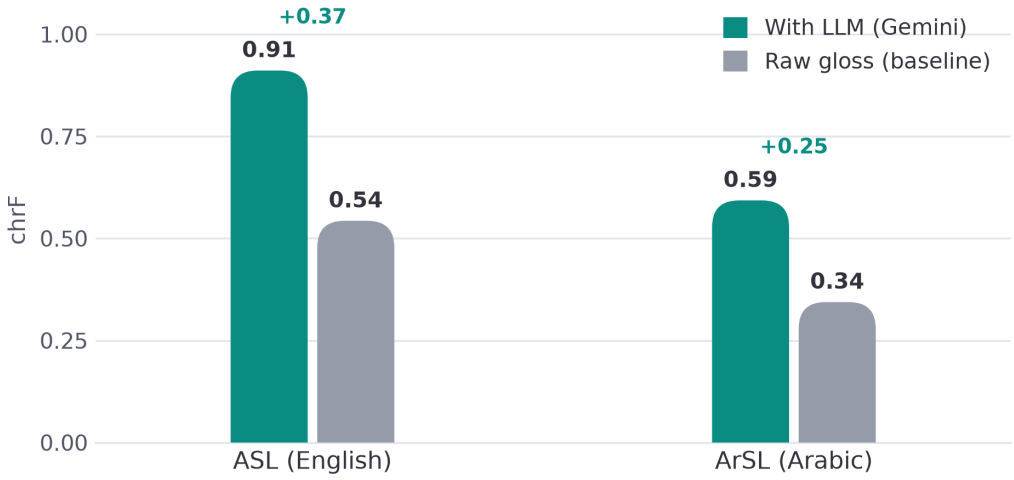

Figure 5.4: Translation quality (chrF): LLM vs. raw gloss

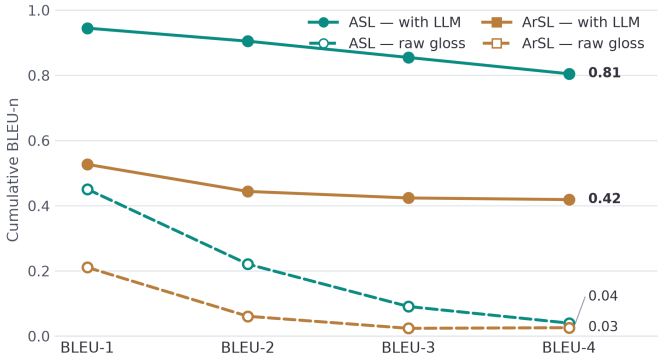

Figure 5.5: Cumulative BLEU-n profiles

5.3.2 Precision–recall analysis

The precision/recall decomposition of chrF (Figure 5.6) localises what the language model adds. For both languages the raw gloss already has moderate precision but poor recall (ASL 0.71/0.52; ArSL 0.54/0.32) — it emits the right content words but omits the function words, particles, and morphology that a grammatical sentence requires. The language model raises recall sharply (ASL to 0.91; ArSL to 0.59), which is exactly the grammatical “glue” identified in Chapter 2.

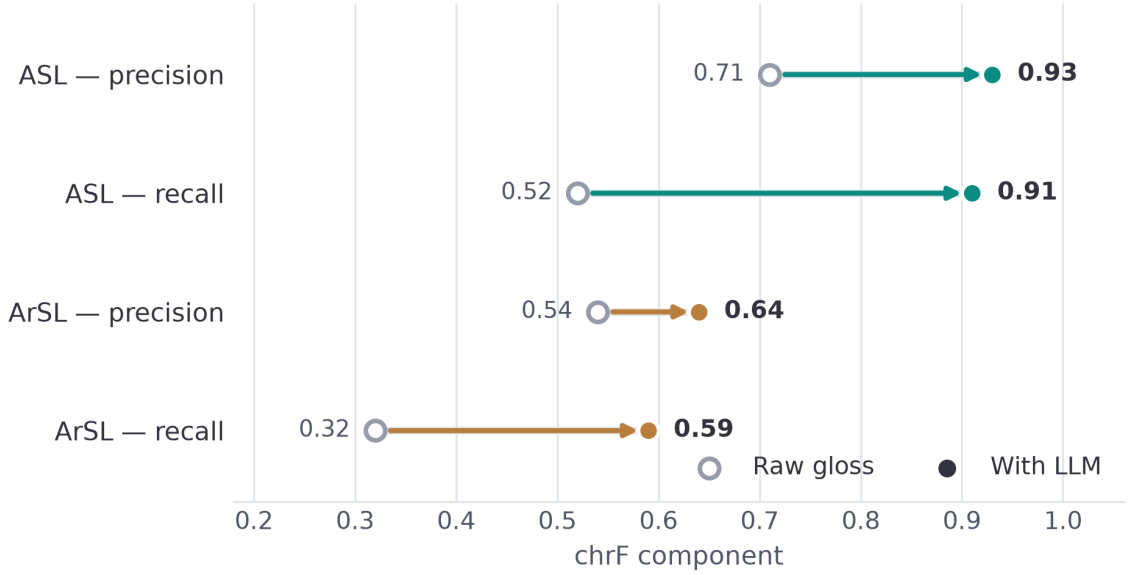

Figure 5.6: chrF precision vs. recall (the LLM supplies grammar = recall)

5.3.3 Discussion

Three findings stand out. First, the improvement from the language model is large and holds across both metrics (for ArSL, chrF rises from 0.34 to 0.59 and BLEU-4 from 0.03 to 0.42), so it is not an artefact of one metric. Second, the gain is concentrated in recall, confirming that the language model supplies grammatical structure rather than merely polishing word choice. Third, the ASL scores exceed the ArSL scores (chrF 0.91 vs 0.59) not because the Arabic system is weaker but because Arabic's rich morphology spreads correct meaning across many valid surface forms, which overlap metrics penalise; this is also why chrF (0.59) is markedly higher than BLEU-4 (0.42) for Arabic and is the fairer measure there.

5.4 System Performance

5.4.1 Latency

Table 5.6: Inference latency of the two recognition models.

Model	Average latency	Max Latency
ASL	43.47 ms	58.7 ms
ArSL	4.21 ms	5.5 ms

5.4.1.1 Measured CPU Micro-benchmarks

Two pipeline stages were profiled directly on CPU. INT8 dynamic quantization of the ArSL predictor's Linear and GRU weights cut inference from 7.48 ms to 4.21 ms per call — a 1.78× speed-up — with no measurable accuracy change. A 256-entry cache keyed on (gloss, language) reduces a repeated gloss-to-sentence call from a cold ~50 ms (and 300–800 ms for a live Gemini round-trip) to effectively zero on a cache hit. The ASL model is executed through the TFLite XNNPACK delegate with up to four threads, accelerating its convolution and dense operations on commodity CPUs.

Table 5.7: Measured CPU micro-benchmarks.

Optimization	Before	After
ArSL predictor — INT8 dynamic quantization	7.48 ms / call	4.21 ms / call (1.78×)
Gloss→sentence — repeat-call cache	50.1 ms (cold)	0.001 ms (cache hit)
Live Gemini round-trip (cached on repeat)	300–800 ms	~0 ms on repeat

5.4.2 Model Quantization and CPU Deployment

A core requirement is that the system run on commodity hardware with no GPU. This is achieved through quantization and lightweight runtimes. The ASL model is exported to TensorFlow Lite and executed through the XNNPACK delegate for fast CPU inference, and the

ArSL PyTorch model is INT8-quantized for CPU execution. Quantization reduces the model footprint and inference time while keeping accuracy effectively unchanged, allowing both models, the SBERT encoder, and the database client to run together within a modest (~2–4 GB) memory budget on a CPU-only server.

5.5 Functional and System Testing

Beyond model accuracy, every part of the platform was tested to confirm it works end to end. Correctness is guarded by two layers. First, an automated pytest suite covers the back-end logic: authentication (Argon2 hashing, JWT issue/refresh, rate limiting), the gloss and non-manual-marker engine, the provider fallback chains, and sentence stitching. Second, a frontend regression module renders every Jinja2 template and asserts the structural invariants of the dashboards — required script blocks present, design-system stylesheets linked, every getElementById target resolvable, the Arabic dashboard right-to-left with no English leakage, and the inline JavaScript parseable. These run under a continuous-integration workflow so regressions are caught on every change. In addition, each user-facing module was manually tested end to end, as summarised in Table 5.8.

Table 5.8: Functional test coverage.

Component	**Test type**	**Verified**
Sign → Text	Manual + integration	✓
Sign → Speech (TTS)	Manual	✓
Text → Sign (avatar)	Manual	✓
Speech → Sign (STT)	Manual	✓
Live two-party meeting (WebRTC)	Manual (two clients)	✓
Authentication (signup / login / refresh)	pytest (auth)	✓
Gloss / non-manual markers	pytest (gloss)	✓

Provider fallback chains	pytest (providers)	✓
Sentence stitching / fingerspelling	pytest (stitch)	✓
Bilingual EN/AR + RTL dashboards	Frontend regression (templates)	✓

5.6 Limitations and Threats to Validity

The evaluation is comparatively thorough — both models are tested for generalization, and the translation gain is isolated against a baseline — but four limitations remain. First, the ASL model is trained on a subset of the full corpus and uses a random in-distribution split; its 80% in-distribution and 62.4% cross-dataset figures leave clear headroom that a larger training set would address. Second, although the ArSL model was evaluated signer-independently (88%), the test set is still a single dataset of phone-camera video, so robustness to very different cameras and environments is unproven. Third, the system recognises isolated signs rather than continuous signing, so the language model — not a continuous recogniser — forms the sentences, and natural conversational signing is not yet supported. Fourth, the Arabic vocabulary is small (20 signs) and its translation BLEU rests on a limited reference set of 20 sentences, so chrF and human judgement are the more trustworthy measures there. These points motivate the future work of Chapter 6.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This thesis set out to address a persistent and practical problem: the communication barrier between Deaf and hard-of-hearing signers and the hearing world, and the absence of tools that bridge it in both directions, for more than one sign language, without specialised hardware. The work delivered Together, a real-time, bidirectional, bilingual sign-language translation platform that runs entirely in a standard web browser and treats both a high-resource language (American Sign Language) and a low-resource one (Arabic/Egyptian Sign Language) as first-class.

The project met each of its objectives. Real-time landmark capture was achieved in the browser with MediaPipe, streaming a compact, privacy-preserving skeleton rather than raw video. Isolated-sign recognition was implemented for both languages with models suited to their data — a Squeezeformer-style 1-D convolution-and-transformer network for the 250-class ASL vocabulary, exported to TFLite, and a compact CNN-GRU for the 20-class ArSL vocabulary in PyTorch. Recognised signs were turned into fluent sentences by a gloss-mediated language stage, and the reverse path animated an avatar from sign clips retrieved by semantic search. The four translation directions were combined into a live, two-party meeting over WebRTC, and the system was finished with the production qualities of authentication, a bilingual right-to-left interface, and a deployable container.

The evaluation supports the central design decision. The ASL model reached 80% accuracy on its own test set and generalised to 62.4% Top-1 cross-dataset on the SignASL benchmark, while the ArSL model reached 99.41% in-distribution and 88% under a signer-independent protocol; and — most importantly for the gloss-mediated approach — the language-model stage improved translation quality substantially and consistently in both languages (chrF rising from 0.54 to 0.91 for ASL and from 0.34 to 0.59 for ArSL), with the gain concentrated in recall, confirming that the language model supplies the grammatical structure that raw gloss lacks. Together therefore

demonstrates that a modular, gloss-mediated, landmark-based design can deliver a complete bidirectional, bilingual system on commodity hardware.

6.2 Contributions

The principal contributions of this work are:

1. A bidirectional, bilingual, browser-based system that translates between sign and spoken language in four directions and supports a live two-party meeting, without gloves, depth cameras, or installation.
2. A working Egyptian Arabic Sign Language recogniser, a contribution to an under-resourced language for which word-level resources are scarce.
3. A gloss-mediated translation design that uses a general-purpose language model to bridge sign-language gloss and spoken-language grammar, removing the dependence on the large parallel signing corpora that end-to-end models require.
4. An engineering blueprint for real-time landmark inference in the browser with graceful offline fallback, reproducible from public datasets.

6.3 Limitations

The results must be read with four limitations in mind. First, although both models were tested for generalization — the ASL model cross-dataset and the ArSL model signer-independently (88%) — the ASL in-distribution split is random rather than signer-independent, so an explicit signer-independent ASL evaluation would further strengthen the claim. Second, the ASL model is trained on a subset of the full corpus, and its accuracy, while strong, leaves clear headroom. Third, the system recognises isolated signs rather than continuous signing, so natural conversational signing is not yet supported. Fourth, the Arabic vocabulary is small (20 signs) and its translation metrics rest on a limited reference set of 20 sentences.

6.4 Future Work

These limitations map directly onto the most valuable directions for future work:

- Signer-independent evaluation for ASL: extend the signer-independent protocol already applied to the ArSL model to the ASL model, for an equally honest measure of generalisation across both languages.
- Larger and more diverse training data: train the ASL model on the full corpus, and expand the ArSL vocabulary well beyond 20 signs to make it practically useful.
- Continuous sign-language recognition: move from isolated signs to continuous signing, allowing natural, sentence-level conversation rather than sign-by-sign accumulation.
- A user study with the Deaf community: evaluate the system with Deaf and hard-of-hearing participants on real communication tasks, measuring comprehension and usability rather than only automatic metrics.
- Latency and deployment hardening: quantify and optimise end-to-end latency (warm models, batched inference, edge deployment) to guarantee interactive performance.
- Broader language coverage: generalise the architecture to additional sign languages, building on the demonstration that it already spans two typologically different ones.

6.5 Closing Remarks

Together shows that an accessible, bidirectional, bilingual sign-language translator is achievable today with commodity hardware and a carefully chosen, modular design. While continuous signing and signer-independent robustness remain open challenges, the system delivers a complete and deployable platform and, in treating Arabic Sign Language as a first-class language, contributes a small but meaningful step toward technology that serves under-represented Deaf communities.

References

[1] World Health Organization, "Deafness and hearing loss," WHO Fact Sheet, 2024. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[2] W. C. Stokoe, "Sign language structure: An outline of the visual communication systems of the American deaf," Studies in Linguistics, Occasional Papers 8, 1960.

[3] R. Rastgoo, K. Kiani, and S. Escalera, "Sign language recognition: A deep survey," Expert Systems with Applications, vol. 164, 2021.

[4] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv preprint arXiv:1906.08172, 2019.

[5] T. B. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877-1901.

[6] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison," in Proc. IEEE Winter Conf. Applications of Computer Vision (WACV), 2020, pp. 1459-1469.

[7] H. R. V. Joze and O. Koller, "MS-ASL: A large-scale data set and benchmark for understanding American sign language," in Proc. British Machine Vision Conf. (BMVC), 2019.

[8] O. M. Sincan and H. Y. Keles, "AUTSL: A large scale multi-modal Turkish sign language dataset and baseline methods," IEEE Access, vol. 8, pp. 181340-181355, 2020.

[9] S. Jiang, B. Sun, L. Wang, Y. Bai, K. Li, and Y. Fu, "Skeleton aware multi-modal sign language recognition," in Proc. IEEE/CVF CVPR Workshops, 2021, pp. 3413-3423.

[10] N. C. Camgoz, S. Hadfield, O. Koller, H. Ney, and R. Bowden, "Neural sign language translation," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 7784-7793.

[11] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, "Sign language transformers: Joint end-to-end sign language recognition and translation," in Proc. IEEE/CVF CVPR, 2020, pp. 10023-10033.

[12] A. Duarte et al., "How2Sign: A large-scale multimodal dataset for continuous American sign language," in Proc. IEEE/CVF CVPR, 2021, pp. 2735-2745.

[13] A. A. I. Sidig, H. Luqman, S. Mahmoud, and M. Mohandes, "KArSL: Arabic sign language database," ACM Trans. Asian and Low-Resource Language Information Processing, vol. 20, no. 1, pp. 1-19, 2021.

[14] G. Latif, N. Mohammad, J. Alghazo, R. AlKhalaf, and R. AlKhalaf, "ArASL: Arabic alphabets sign language dataset," Data in Brief, vol. 23, 2019.

[15] M. Mohandes, M. Deriche, and J. Liu, "Image-based and sensor-based approaches to Arabic sign language recognition," IEEE Trans. Human-Machine Systems, vol. 44, no. 4, pp. 551-557, 2014.

[16] W. Sandler and D. Lillo-Martin, Sign Language and Linguistic Universals. Cambridge, U.K.: Cambridge Univ. Press, 2006.

[17] S. K. Liddell, Grammar, Gesture, and Meaning in American Sign Language. Cambridge, U.K.: Cambridge Univ. Press, 2003.

[18] SignAll Technologies, "SignAll," 2024. [Online]. Available: https://SignAll.us

[19] Hand Talk, "Hand Talk Translator," 2024. [Online]. Available: https://www.handtalk.me

[20] SLAIT, "SLAIT - Real-time sign language translation," 2023. [Online]. Available: https://www.slait.ai

[21] KinTrans, "KinTrans - AI that understands sign language," 2024. [Online]. Available: https://www.kintrans.com

[22] Signapse AI, "Signapse - AI sign language translation," 2024. [Online]. Available: https://www.signapse.ai

[23] Google, "Google - Isolated Sign Language Recognition," Kaggle Dataset, 2023. [Online]. Available: https://www.kaggle.com/competitions/asl-signs

[24] H. M. Balaha, "ASL 20 Words Dataset (v1)," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/hossammbalaha/asl-20-words-dataset-v1

[25] F. Zhang et al., "MediaPipe Hands: On-device real-time hand tracking," arXiv preprint arXiv:2006.10214, 2020.

[26] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992.

[27] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023.

[28] H. Touvron et al., "LLaMA: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023.

[29] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[30] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2014, pp. 1724-1734.

[31] C. Jennings, H. Bostrom, and J.-I. Bruaroey, Eds., "WebRTC 1.0: Real-time communication between browsers," W3C Recommendation, 2021.

[32] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in Proc. 40th Annu. Meeting Assoc. for Computational Linguistics (ACL), 2002, pp. 311-318.

[33] M. Popovic, "chrF: Character n-gram F-score for automatic MT evaluation," in Proc. 10th Workshop on Statistical Machine Translation (WMT), 2015, pp. 392-395.

[34] M. Post, "A call for clarity in reporting BLEU scores," in Proc. 3rd Conf. Machine Translation (WMT), 2018, pp. 186-191.